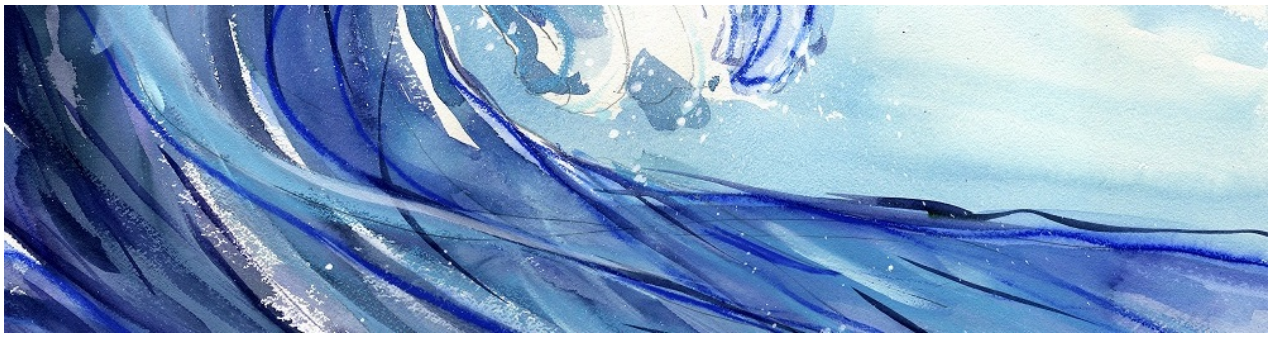
INFORMATION IMAGES APPRENTISSAGE

Une introduction aux sciences des données

Gabriel Peyré

CNRS & DMA
École normale supérieure, Université PSL

9 septembre 2026



Une introduction aux idées mathématiques qui permettent de représenter, traiter et apprendre à partir des données.

https://mathematical-tours.github.io/book-basics/

Présentation

Ce livre propose cinq introductions à des idées mathématiques qui permettent de représenter, de traiter et d'apprendre à partir des données. Les exemples concrets servent de point de départ, tandis que des énoncés précis expliquent ce qu'un algorithme peut ou ne peut pas garantir.

- Le chapitre 1 introduit le codage binaire et la borne entropique de Shannon pour la compression sans perte.
- Le chapitre 2 considère une image comme un tableau de nombres et développe la quantification, le filtrage, la détection de contours et les transformations de couleurs.
- Le chapitre 3 relie l'échantillonnage et l'approximation parcimonieuse à la régularisation des problèmes inverses et à l'acquisition comprimée.
- Le chapitre 4 développe le transport optimal, des affectations finies à la comparaison et à l'interpolation de distributions.
- Le chapitre 5 explique les réseaux de neurones discriminatifs et génératifs, leurs objectifs d'apprentissage et leur lien avec le transport.

Les deux premiers chapitres demandent des connaissances d'algèbre élémentaire et, pour la théorie de l'information, de logarithmes et de probabilités de base. Les suivants utilisent les vecteurs, les matrices, les dérivées et les distributions de probabilité. Les résultats les plus avancés sont énoncés avec leurs hypothèses ; les exemples qui les entourent peuvent se lire avant d'étudier ces résultats en détail.

Chaque chapitre peut être lu séparément. Une lecture suivie fait apparaître des liens utiles : l'entropie explique le codage, les transformées rendent les signaux compressibles, la régularisation sélectionne les reconstructions et le transport compare les distributions qu'un modèle génératif cherche à apprendre. Une notation commune et de courtes introductions de chapitres accompagnent ces deux parcours.

Les éditions française et anglaise contiennent les mêmes cinq chapitres. Les livres complets, les chapitres séparés et les sources sont disponibles à l'adresse

https://mathematical-tours.github.io/book-basics/

Pour un traitement plus approfondi, on pourra consulter *Mathematical Foundations of Data Sciences*, disponible sur https://mathematical-tours.github.io/book/.

Remerciements. Je remercie Marie-Noëlle Peyré, Gwenn Guichaoua, François Béguin, Gérard Grancher, Aurélien Djament, François Sauvageot, Charles Dossal, Jalal Fadili, Samuel Vaiter, Stéphane Seuret, Vincent Beck, Sébastien Racanière, Vincent Barra et le rapporteur anonyme de l'article original sur la parcimonie pour leurs relectures attentives et leurs corrections. La photographie de la fleur est de Maïtine Bergounioux.

Gabriel Peyré
CNRS & DMA, École normale supérieure, PSL

Table des matières

Présentation **ii**

1 Claude Shannon et la compression des données **1**
1.1 Données numériques et codage . . . 1
1.2 Codage et décodage . . . 1
1.3 Entropie et borne de Shannon . . . 3

2 Le traitement d'images **7**
2.1 Pixels et niveaux de gris . . . 7
2.2 Stocker une image . . . 7
2.3 Réduire le bruit . . . 9
2.4 Détecter les contours . . . 10
2.5 Images en couleur . . . 10
2.6 Modifier la luminosité et le contraste . . . 12
2.7 Images et matrices . . . 13

3 Parcimonie, problèmes inverses et acquisition comprimée **15**
3.1 Du signal continu aux échantillons . . . 15
3.2 Approximation non linéaire et compression . . . 16
3.3 Problèmes inverses et régularisation . . . 17
3.4 Acquisition comprimée . . . 19

4 Transport optimal **21**
4.1 Le problème de Monge . . . 21
4.2 La relaxation de Kantorovitch . . . 22
4.3 Mesures et distances de transport . . . 24
4.4 Applications . . . 25

5 Réseaux de neurones **27**
5.1 Réseaux discriminatifs . . . 27
5.2 Réseaux génératifs . . . 30

Glossaire **33**

Références **34**

CHAPITRE 1

Claude Shannon et la compression des données

Combien de bits faut-il pour conserver un message sans perdre d'information ? La théorie de Shannon relie cette question aux probabilités des symboles qui composent le message. À partir d'une petite image en niveaux de gris, nous introduisons les codes binaires, expliquons le décodage sans ambiguïté et précisons le rôle de l'entropie dans la compression.

1.1 Données numériques et codage

Textes, sons, images et vidéos sont stockés et transmis sous forme de suites de bits : des chiffres binaires 0 et 1. Avant de choisir cette représentation, on décrit les données par des *symboles*, par exemple les lettres d'un texte ou les valeurs des pixels d'une image. Une méthode de compression sans perte remplace une suite de symboles par une représentation binaire plus courte dont on peut retrouver exactement la suite initiale.

Les travaux de Shannon [41, 42] ont établi des limites fondamentales pour la représentation et la transmission de l'information. Il convient de distinguer plusieurs opérations :

- L'*échantillonnage* relève les valeurs d'un signal continu en certains points ou instants. La *quantification* remplace ensuite ces valeurs par des éléments d'un ensemble fini.
- Le *codage de source*, étudié dans ce chapitre, recherche une représentation compacte des symboles obtenus.
- Le *codage de canal* ajoute de la redondance pour détecter ou corriger des erreurs de transmission. La protection contre des modifications malveillantes relève d'un problème de sécurité distinct.

L'échantillonnage et la quantification peuvent supprimer de l'information. Le codage étudié ici est sans perte par rapport aux symboles déjà acquis. Cette distinction est essentielle lorsque l'on parle de compresser une photographie ou un enregistrement sonore.

1.2 Codage et décodage

1.2.1 Une petite image en niveaux de gris

Une image en niveaux de gris est un tableau de pixels. Pour simplifier, considérons quatre valeurs possibles : 0 pour le noir, 1 pour le gris foncé, 2 pour le gris clair et 3 pour le blanc. Les mêmes idées s'appliquent à un alphabet plus grand, comme les 256 valeurs d'une image en niveaux de gris sur huit bits, ainsi qu'aux composantes d'une image en couleur.

La figure 1.1 représente une fleur et l'agrandissement d'un carré de 5×5 pixels. En lisant ses 25 valeurs dans l'ordre indiqué, on obtient

$$(0,1,3,2,0,3,2,2,1,2,2,1,1,2,2,2,1,1,2,2,2,1,1,2,1). \tag{1.1}$$

1.2.2 Codage à longueur fixe

Sauf si la longueur du message est connue à l'avance, on impose des mots de code non vides. Un *mot de code* est la suite de bits associée à un symbole. On peut choisir

$$0 \mapsto 00, \qquad 1 \mapsto 01, \qquad 2 \mapsto 10, \qquad 3 \mapsto 11.$$

Les quatre mots ont une longueur égale à deux : il s'agit d'un code *à longueur fixe*, également appelé code uniforme. Coder un message consiste à concaténer les mots de code de ses symboles. Par exemple,

$$(0,1,3) \longmapsto (00,01,11) \longmapsto 000111.$$

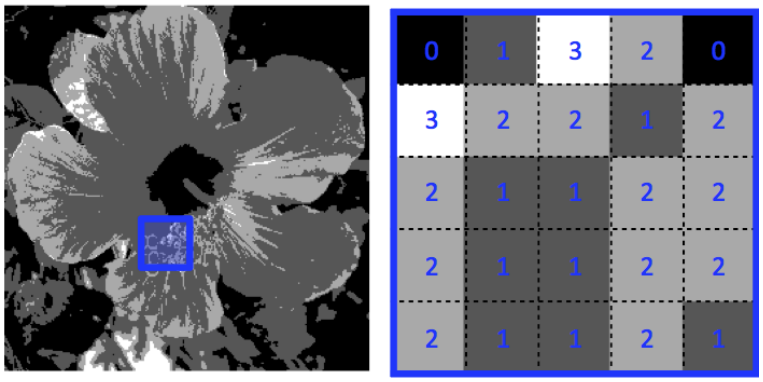


Figure 1.1. Une image à quatre niveaux de gris et l'agrandissement d'un carré de 5×5 pixels.

Le message complet (1.1) devient

$$\texttt{0001111000111010011010010110101001011010100101100 1}.$$

Sa longueur totale vaut $\overline{L} = 25 \times 2 = 50$ bits. Ce calcul porte sur les valeurs des pixels codées ; un fichier peut aussi contenir les dimensions de l'image, une table de codage et d'autres métadonnées.

Pour un alphabet de $N \geq 2$ symboles, un code binaire à longueur fixe ℓ existe exactement lorsque $2^\ell \geqslant N$: il faut disposer d'au moins autant de mots binaires distincts que de symboles. La plus petite longueur possible est donc

$$\ell = \lceil \log_2 N \rceil.$$

Lorsque $N = 2^\ell$, tous les mots binaires sont utilisés. Pour $N = 8$, par exemple, ce sont 000, 001, 010, 011, 100, 101, 110, 111, de longueur $\ell = 3$.

Le logarithme en base deux est défini pour $x > 0$ par $\log_2 x = \ln x / \ln 2$. Il vérifie $\log_2(1/x) = -\log_2 x$; en particulier,

$$\log_2(1/4) = -\log_2 4 = -2.$$

L'arrondi à l'entier supérieur dans la formule de ℓ est indispensable lorsque N n'est pas une puissance de deux.

1.2.3 Codage à longueur variable

On peut réduire le nombre moyen de bits en attribuant des mots plus courts aux symboles fréquents. Considérons le code

$$0 \mapsto 001, \qquad 1 \mapsto 01, \qquad 2 \mapsto 1, \qquad 3 \mapsto 000. \tag{1.2}$$

La suite $(0, 1, 3)$ devient 00101000 et le message complet s'écrit

$$\texttt{0010100010010001101110101111010111101011101}.$$

Cette suite comporte 42 bits. Sa *longueur moyenne par symbole* vaut

$$L = \frac{\overline{L}}{25} = \frac{42}{25} = 1{,}68 \text{ bits}.$$

Pour ce message, le code est plus économique que le code à longueur fixe. Il ne l'est pas nécessairement pour tout message : une suite composée surtout des symboles 0 et 3 demanderait près de trois bits par symbole.

1.2.4 Codes préfixes et décodage sans ambiguïté

Un code est *uniquement décodable* si deux suites finies de symboles différentes ne produisent jamais la même suite de bits. Un code à longueur fixe dont les mots sont distincts possède cette propriété : il suffit de découper le message en blocs de ℓ bits, puis de lire la table de codage en sens inverse.

Pour un code à longueur variable, il ne suffit pas que les mots soient distincts. Ainsi, le code

$$0 \mapsto 0, \quad 1 \mapsto 10, \quad 2 \mapsto 110, \quad 3 \mapsto 101$$

est ambigu : 1010 = 10 10 = 101 0 représente aussi bien $(1, 1)$ que $(3, 0)$.

Une condition suffisante pour assurer l'unicité du décodage est qu'aucun mot ne soit un préfixe propre d'un autre. On parle alors de *code préfixe*. Le code (1.2) vérifie cette condition. Chaque mot est reconnu dès que son dernier bit est lu :

$$\underbrace{001}_{0}\ \underbrace{01}_{1}\ \underbrace{000}_{3}\ \underbrace{1}_{2}\ \cdots.$$

Les espaces servent ici à expliquer le décodage ; ils ne sont pas transmis.

La condition de préfixe est suffisante, mais pas nécessaire. Le code $A \mapsto 0$, $B \mapsto 01$ est uniquement décodable, bien que 0 soit un préfixe de 01. Chaque 1 doit être associé au 0 qui le précède : le message complet ne possède donc qu'un seul découpage. Après avoir lu un 0, le décodeur doit néanmoins regarder la suite pour décider s'il représente A ou commence B. D'autres codes non préfixes peuvent imposer des décisions différées ou des retours en arrière ; les codes préfixes évitent cette difficulté.

1.2.5 Codes et arbres binaires

Le mot vide est la racine d'un arbre binaire. Ajouter un 0 ou un 1 revient à choisir son fils gauche ou son fils droit. Un mot de longueur ℓ se trouve à la profondeur ℓ ; l'arbre complet de profondeur ℓ possède donc 2^ℓ feuilles et $\ell + 1$ niveaux, racine comprise.

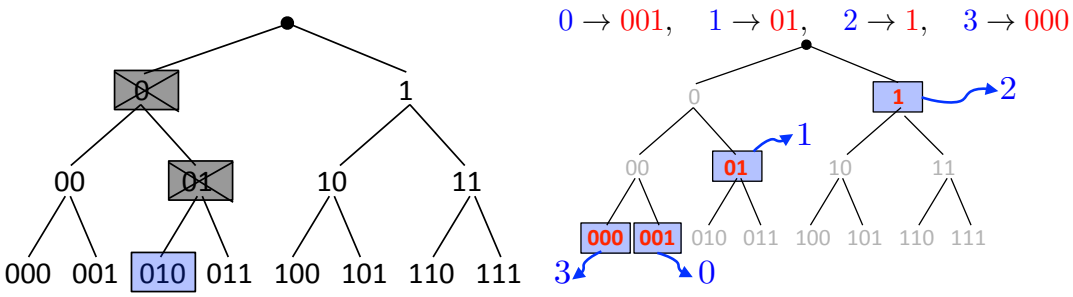


Figure 1.2. L'arbre binaire complet de profondeur trois (à gauche) et le sous-arbre d'un code préfixe (à droite).

Dans un code préfixe, aucun nœud choisi comme mot de code n'est l'ancêtre d'un autre. Les nœuds choisis sont donc les feuilles de l'arbre formé par leurs chemins depuis la racine. Pour décoder, on part de la racine et l'on suit une arête par bit lu. Quand on atteint une feuille portant un mot de code, on émet le *symbole* correspondant, puis on revient à la racine. Cette procédure rend le décodage immédiat et sans ambiguïté.

1.3 Entropie et borne de Shannon

1.3.1 Un modèle probabiliste

La qualité d'un code dépend des messages auxquels on l'applique. Un modèle probabiliste permet d'expliciter cette dépendance. Soit V un symbole aléatoire à valeurs dans $\{0, \ldots, N-1\}$, de probabilités

$$p_v = \mathbb{P}(V = v), \qquad p_v \geqslant 0, \qquad \sum_{v=0}^{N-1} p_v = 1.$$

Le modèle le plus simple produit une suite de symboles indépendants ayant tous cette distribution. L'indépendance est une hypothèse sur la source, et non une propriété assurée par l'algorithme de codage.

Pour un message observé, on peut estimer les probabilités par les *fréquences empiriques*. Le message (1.1) contient 2 zéros, 9 uns, 12 deux et 2 trois. Sa distribution empirique est

$$p = (2/25,\ 9/25,\ 12/25,\ 2/25).$$

Utiliser ces fréquences comme probabilités constitue un choix de modélisation. Si le décodeur ne connaît pas déjà le modèle ou la table de codage retenus, il faut également transmettre les informations qui les décrivent.

1.3.2 Entropie

L'*entropie* de V, mesurée en bits, est

$$H(V) = -\sum_{v=0}^{N-1} p_v \log_2 p_v. \tag{1.3}$$

Un symbole de probabilité $p_v > 0$ apporte une information égale à $-\log_2 p_v$. L'entropie est la moyenne de cette quantité. La convention $0 \log_2 0 = 0$ signifie que les symboles impossibles ne contribuent pas à la somme ; elle est cohérente avec $\lim_{x \downarrow 0} x \log_2 x = 0$.

Comme $-\log_2 p_v \geqslant 0$, l'entropie est positive ou nulle. Plus précisément,

$$0 \leqslant H(V) \leqslant \log_2 N.$$

Les valeurs extrêmes décrivent deux modèles utiles :

- **Aucune incertitude.** Si $p_1 = 1$, tous les symboles valent 1 et $H(V) = 0$. Le message est donc une suite de uns.
- **Incertitude maximale.** Si $p_v = 1/N$ pour tout v, alors $H(V) = \log_2 N$. Avec quatre symboles équiprobables, cette valeur est de deux bits. Un message aléatoire produit par ce modèle ne contient pas nécessairement autant d'occurrences de chaque symbole.
- **Une situation intermédiaire.** Pour la distribution empirique précédente,

$$H(V) = -2\frac{2}{25}\log_2\frac{2}{25} - \frac{9}{25}\log_2\frac{9}{25} - \frac{12}{25}\log_2\frac{12}{25} \simeq 1{,}6219 \text{ bits}.$$

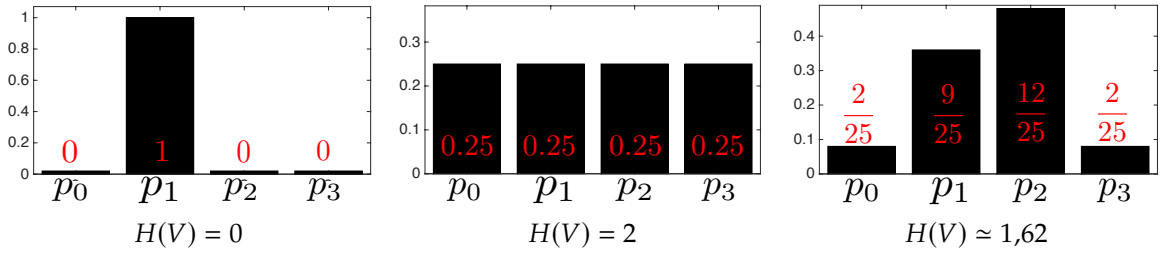


Figure 1.3. Une distribution déterministe, la distribution uniforme sur quatre symboles et la distribution empirique du message de 25 pixels.

1.3.3 Longueur moyenne d'un code

Notons c_v le mot de code du symbole v et ℓ_v sa longueur. Le nombre moyen de bits par symbole est

$$L(c) = \mathbb{E}[\ell_V] = \sum_v p_v \ell_v.$$

Pour le code (1.2), les longueurs sont $(3, 2, 1, 3)$, d'où

$$L(c) = \frac{2\times 3 + 9\times 2 + 12\times 1 + 2\times 3}{25} = \frac{42}{25} = 1{,}68.$$

Si p est la distribution empirique d'un message fini, cette formule donne aussi sa longueur moyenne exacte. Si p décrit un modèle de source, il s'agit d'une espérance.

1.3.4 La borne inférieure de l'entropie

Tout code binaire préfixe vérifie la borne de Shannon [41]

$$H(V) \leqslant L(c). \tag{1.4}$$

En voici une courte preuve. Soit m la plus grande longueur d'un mot de code. Un mot de longueur ℓ_v possède $2^{m-\ell_v}$ descendants à la profondeur m. Pour les mots d'un code préfixe, ces ensembles de descendants sont disjoints, donc

$$\sum_v 2^{m-\ell_v} \leqslant 2^m, \qquad Z \overset{\text{def.}}{=} \sum_v 2^{-\ell_v} \leqslant 1.$$

C'est l'inégalité de Kraft. Posons $q_v = 2^{-\ell_v}/Z$. Alors $\sum_v q_v = 1$ et $\ell_v = -\log_2 q_v - \log_2 Z$, ce qui donne

$$\begin{aligned} L(c) - H(V) &= \sum_{v : p_v > 0} p_v \log_2 \frac{p_v}{q_v} - \log_2 Z \\ &\geqslant \sum_{v : p_v > 0} p_v \log_2 \frac{p_v}{q_v} \geqslant 0. \end{aligned}$$

La dernière somme est la divergence de Kullback–Leibler. Sa positivité découle de $\ln t \leqslant t - 1$: en appliquant cette inégalité à $t = q_v/p_v$ puis en sommant, on obtient $\sum p_v \ln(q_v/p_v) \leqslant \sum_{p_v>0} q_v - 1 \leqslant 0$.

La borne porte sur une *moyenne* ; elle n'impose pas séparément $\ell_v \geqslant -\log_2 p_v$ pour chaque symbole. Les nombres réels $-\log_2 p_v$ fournissent néanmoins des longueurs cibles utiles : une probabilité *plus grande* donne une longueur cible plus petite. Pour une distribution uniforme sur 2^ℓ symboles, le code à longueur fixe atteint exactement la borne. Pour d'autres tailles d'alphabet, l'intégralité des longueurs peut empêcher l'égalité.

1.3.5 Codage de Huffman et codage par blocs

L'algorithme de Huffman [23] construit un code préfixe de longueur moyenne minimale pour une distribution finie donnée. Les symboles de probabilité nulle sont omis de l'alphabet effectif. Pour une source ayant au moins deux symboles de probabilité strictement positive, cette longueur vérifie

$$H(V) \leqslant L(c) < H(V) + 1.$$

On peut comprendre la borne supérieure en arrondissant chaque longueur cible à l'entier supérieur : $\ell_v = \lceil -\log_2 p_v \rceil$. Ces longueurs vérifient l'inégalité de Kraft et peuvent être réalisées par un code préfixe. Leur moyenne est strictement inférieure à $H(V) + 1$, et le code de Huffman fait au moins aussi bien.

Pour réduire le surcoût par symbole, on peut coder des blocs de n symboles indépendants. Leur entropie vaut $H(V_1, \dots, V_n) = nH(V)$. Un code par blocs approprié a donc une longueur moyenne L_n telle que

$$H(V) \leqslant \frac{L_n}{n} < H(V) + \frac{1}{n}.$$

Le débit tend ainsi vers l'entropie lorsque la taille des blocs augmente. Le codage arithmétique [36] met en œuvre des idées voisines sans mémoriser une table de mots de code de taille exponentielle. Pour les messages finis, il subsiste des surcoûts liés au modèle, à la terminaison et à la précision numérique.

Si une source n'admet qu'un seul symbole, son entropie est nulle. Lorsque ce symbole et la longueur du message sont connus, aucun bit de contenu n'est nécessaire. Imposer un mot de code non vide coûterait au contraire au moins un bit par symbole. Il faut donc préciser les conventions de délimitation du message et l'alphabet effectivement utilisé.

1.3.6 Exploiter la dépendance entre pixels

Les pixels voisins d'une image sont généralement dépendants. Un modèle qui traite les symboles séparément ignore cette structure. Une transformation des données avant le codage permet souvent de mieux l'exploiter.

Indexons les P pixels par $v_0, \dots, v_{P-1}$ et définissons

$$d_i = v_i - v_{i-1}, \qquad 1 \leqslant i \leqslant P - 1.$$

Conservons v_0 ainsi que les différences. La transformation

$$(v_0, \dots, v_{P-1}) \longmapsto (v_0, d_1, \dots, d_{P-1})$$

est bijective, car $v_i = v_0 + \sum_{j=1}^{i} d_j$. Dans une région presque constante, la plupart des différences sont nulles ou petites. Pour des pixels à quatre niveaux, elles appartiennent à $\{-3, -2, -1, 0, 1, 2, 3\}$.

Pour l'image de la fleur, l'histogramme des différences est plus concentré que celui des pixels. Les entropies empiriques et les longueurs de Huffman de la figure 1.5 illustrent le gain obtenu en codant séparément les valeurs. Ce gain n'est pas garanti pour toute image.

Le code représenté est

d	-3	-2	-1	0	1	2	3
c_d	010101	01011	011	1	00	0100	010100

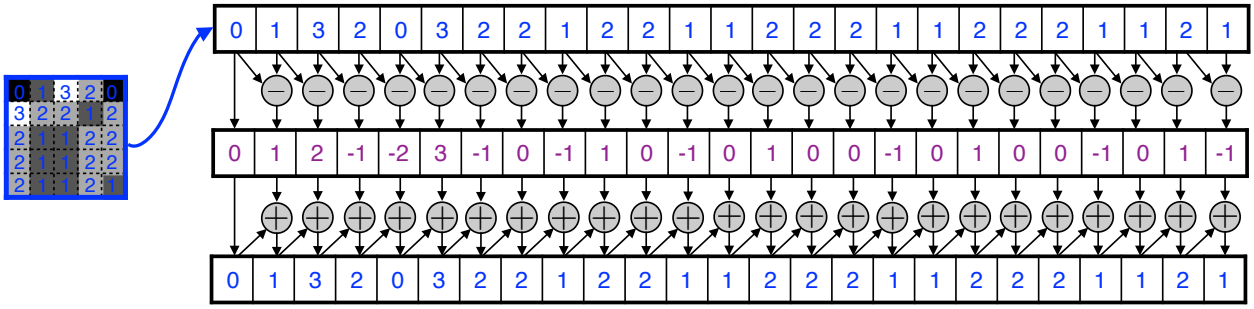


Figure 1.4. Différences successives et reconstruction par sommation cumulée, en conservant le premier pixel.

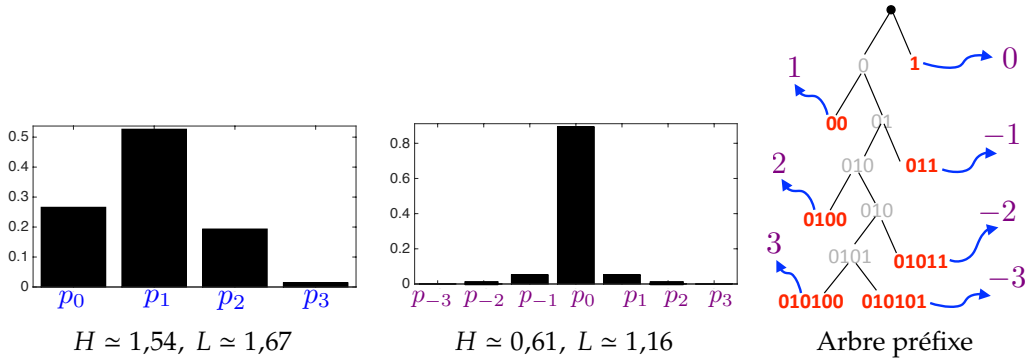


Figure 1.5. Histogrammes des pixels et des différences sur l'image entière, avec un arbre de codage préfixe des différences.

et sa longueur moyenne est d'environ 1,16 bit par différence. En utilisant les débits arrondis affichés pour une image de 256×256 pixels, on estime le contenu codé à 16 Kio pour le code uniforme sur deux bits, 13,36 Kio pour le codage direct des pixels et 9,28 Kio pour le codage des différences. Un Kio vaut 1024 octets, soit 8192 bits. Il faut ajouter à la dernière estimation le premier pixel et la description de la table de codage.

Une transformation bijective ne réduit *pas* l'entropie jointe du message entier. Elle peut rendre les modèles symbole par symbole plus efficaces en modifiant les distributions marginales et les dépendances. La borne (1.4) ne requiert aucune hypothèse d'indépendance pour un symbole aléatoire pris isolément ; cette hypothèse a servi plus haut à obtenir $H(V_1, \ldots, V_n) = nH(V)$. Pour une source dépendante, c'est l'entropie jointe du bloc qui fournit la borne pertinente.

Les méthodes de compression d'images plus élaborées combinent prédiction ou transformations, quantification et codage entropique. JPEG 2000, par exemple, utilise les ondelettes et possède des modes avec et sans perte [29]. L'exploitation des dépendances s'applique également aux lettres d'un texte.

Pour aller plus loin. On pourra consulter [12] pour approfondir le codage de source et [29] pour ses applications au signal et à l'image. Les programmes des figures originales sont disponibles sur https://github.com/gpeyre/2016-shannon-theory ; d'autres expériences figurent dans les Numerical Tours [34].

CHAPITRE 2

Le traitement d'images

Une image numérique est un tableau de nombres : la modifier revient donc à calculer avec ces nombres. Ce chapitre présente des méthodes simples pour stocker les images, réduire leur bruit, détecter les contours et manipuler les couleurs. Chaque exemple relie un effet visible à une formule explicite et précise ce que l'opération conserve ou fait perdre.

2.1 Pixels et niveaux de gris

Une image en niveaux de gris ayant n lignes et p colonnes contient np *pixels*. Dans la représentation sur huit bits étudiée ici, chaque pixel contient un entier compris entre 0 et $255 = 2^8 - 1$. Zéro est affiché en noir, 255 en blanc et les valeurs intermédiaires en nuances de gris. D'autres profondeurs de codage et des représentations en nombres flottants sont également utilisées.

La fleur de la figure 2.1 comporte $256 \times 256 = 65\,536$ pixels. L'agrandissement montre un carré de 5×5 pixels : chaque nuance affichée correspond à l'un des nombres du tableau. Une photographie peut contenir des millions de pixels, mais le principe reste le même.

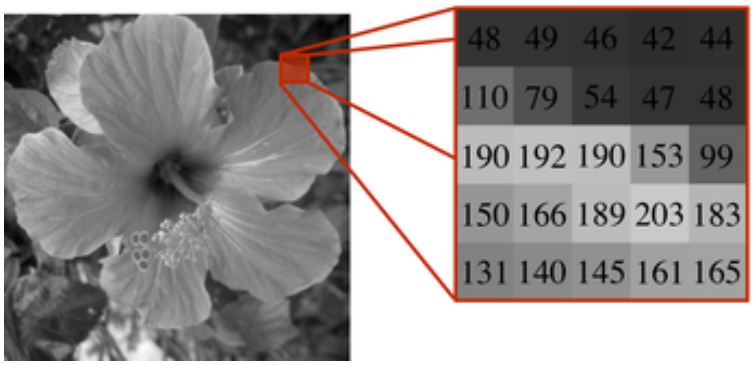


Figure 2.1. Une image et l'agrandissement d'un carré de 5×5 valeurs de pixels.

2.2 Stocker une image

2.2.1 Écriture binaire et unités de stockage

L'écriture binaire d'un entier s'obtient en le décomposant en puissances de deux. Par exemple,

$$179 = 2^7 + 2^5 + 2^4 + 2^1 + 2^0 = \underbrace{(10110011)_2}_{\text{écriture binaire}}.$$

En allant de 2^7 à 2^0, on inscrit un bit égal à un si la puissance correspondante apparaît, et zéro sinon. Huit bits suffisent pour représenter tous les entiers de 0 à 255.

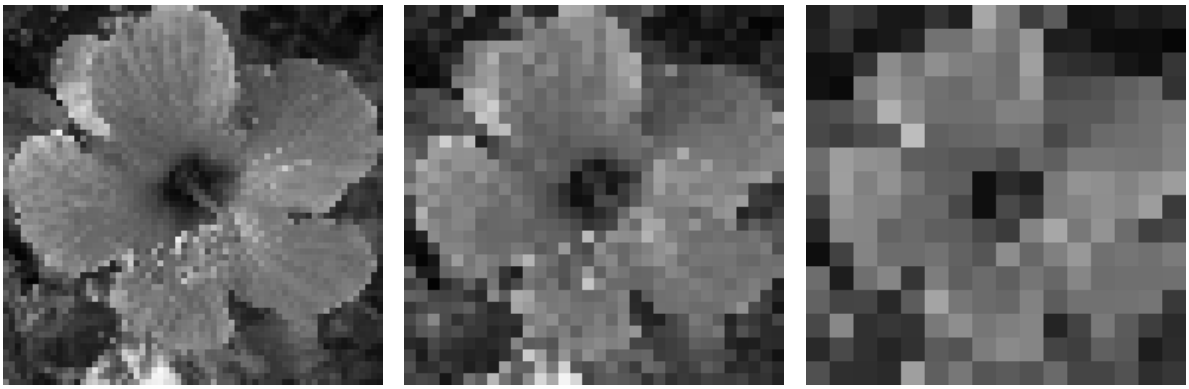
Une image en niveaux de gris sur huit bits non compressée demande donc $8np$ bits, soit np *octets*, pour les valeurs des pixels. La fleur de 256×256 pixels occupe

$$256 \times 256 \times 8 = 524\,288 \text{ bits} = 65\,536 \text{ octets}.$$

Cela représente 65,536 ko en unités décimales, ou 64 Kio en unités binaires : un ko vaut 1000 octets, tandis qu'un Kio vaut 1024 octets. À titre de comparaison, une image de 240×240 pixels demanderait 57 600 octets, soit 56,25 Kio. Les en-têtes de fichier et les métadonnées s'ajoutent à ces volumes.

2.2.2 Sous-échantillonnage

Réduire le nombre de pixels diminue le volume de stockage. Conserver une ligne sur quatre et une colonne sur quatre divise ce nombre par $4^2 = 16$, lorsque les deux dimensions sont divisibles par quatre. La figure 2.2 montre des grilles de plus en plus grossières.



Une ligne/colonne sur 4 Une ligne/colonne sur 8 Une ligne/colonne sur 16

Figure 2.2. Sous-échantillonnage sur des grilles de plus en plus grossières.

Supprimer des lignes et des colonnes peut effacer des détails et créer de faux motifs : c'est le phénomène de *repliement spectral*, ou aliasing. En pratique, on lisse généralement l'image avant de réduire sa fréquence d'échantillonnage. Le chapitre suivant explique la raison mathématique de cette précaution.

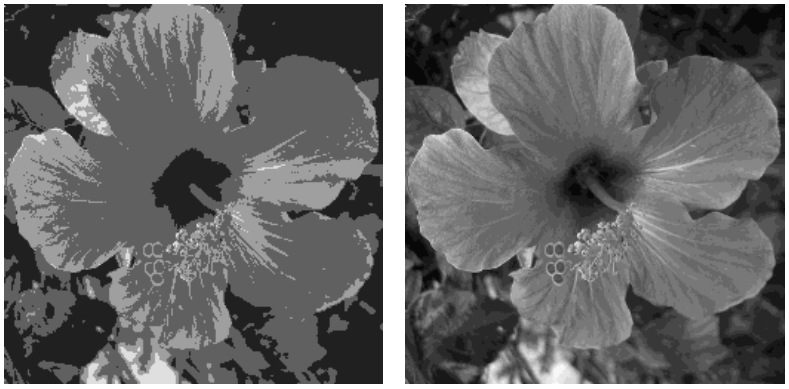
2.2.3 Quantification

On peut aussi réduire le stockage en diminuant le nombre de valeurs possibles par pixel. Pour ne conserver que quatre niveaux, on remplace une valeur $a \in \{0, \ldots, 255\}$ par

$$q(a) = \left\lfloor \frac{a}{64} \right\rfloor \in \{0, 1, 2, 3\}.$$

Ainsi, les valeurs de 0 à 63 deviennent 0, celles de 64 à 127 deviennent 1, celles de 128 à 191 deviennent 2, et celles de 192 à 255 deviennent 3. Affichés sur une échelle du noir au blanc, ces indices correspondent respectivement au noir, au gris foncé, au gris clair et au blanc ; on peut les restituer par les valeurs $85q(a)$.

Quatre indices demandent deux bits au lieu de huit : le gain est donc d'un facteur quatre si ces indices sont effectivement regroupés dans le fichier. Les conserver dans des emplacements de huit bits ne produirait pas ce gain. La figure 2.3 compare quatre et seize niveaux.



4 niveaux de gris 16 niveaux de gris

Figure 2.3. Quantification avec deux nombres de niveaux différents.

La quantification est généralement irréversible : plusieurs valeurs initiales partagent le même indice. La compression sans perte conserve au contraire toutes les valeurs tout en exploitant les redondances de leur

représentation. JPEG 2000 utilise les ondelettes et propose des modes avec et sans perte ; c'est un exemple de codec, et non une méthode universellement meilleure pour toute image et tout critère. [1]

2.3 Réduire le bruit

2.3.1 Moyenne locale

Le bruit produit des fluctuations indésirables des valeurs des pixels. Il est particulièrement visible dans les photographies prises en faible lumière, mais sa distribution dépend du capteur et du procédé d'acquisition. Une méthode simple de débruitage remplace chaque pixel par la moyenne d'une fenêtre autour de lui.

Pour le voisinage de 3×3 pixels de la figure 2.4, on remplace la valeur centrale a par

$$\frac{a+b+c+d+e+f+g+h+i}{9}.$$

Les valeurs affichées donnent

$$\frac{190+192+79+54+47+153+203+189+166}{9} = \frac{1273}{9} \simeq 141{,}44.$$

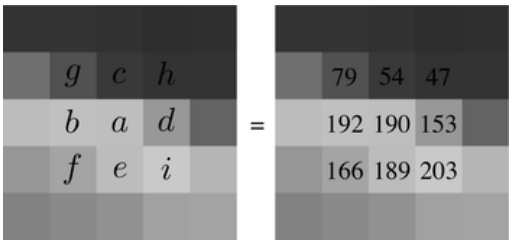


Figure 2.4. Un voisinage de 3×3 pixels pour le calcul d'une moyenne ou d'une médiane locale.

Toutes les moyennes doivent être calculées à partir de la même image d'entrée. Modifier les pixels sur place, puis réutiliser ces nouvelles valeurs, définit un autre filtre. Au bord de l'image, il faut également choisir une convention, par exemple prolonger l'image par réflexion ou restreindre la fenêtre en renormalisant ses poids.

Pourquoi la moyenne est-elle utile ? Si un signal localement constant est perturbé par un bruit indépendant, centré et de variance σ^2, la moyenne de m pixels a une variance de bruit égale à σ^2/m. Près d'un contour, le signal n'est cependant pas constant : la moyenne mélange des régions distinctes et introduit du flou. Agrandir la fenêtre réduit davantage le bruit, mais efface plus de détails.

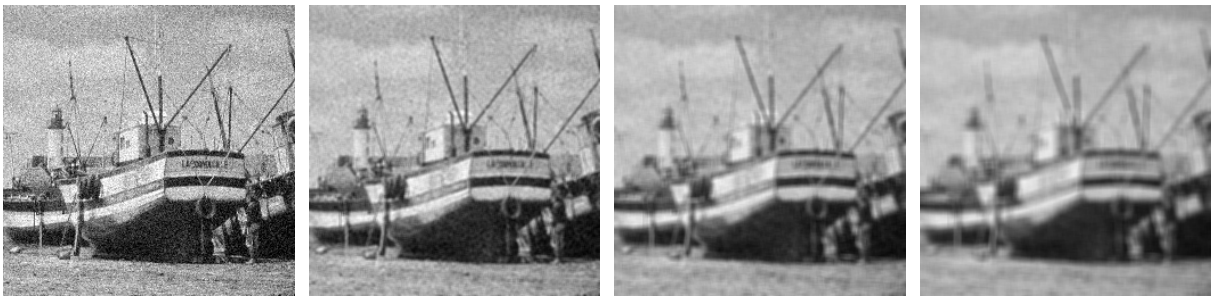


Image bruitée | Moyenne 3×3 | Moyenne 5×5 | Moyenne 7×7

Figure 2.5. La moyenne locale réduit le bruit, mais rend les contours plus flous lorsque la fenêtre grandit.

2.3.2 Médiane locale

La *médiane* d'un nombre impair de valeurs est la valeur centrale après leur classement. Les neuf valeurs précédentes, dans l'ordre croissant, sont

$$47, 54, 79, 153, 166, 189, 190, 192, 203.$$

1. https://jpeg.org/jpeg2000/

Leur médiane est donc 166. Le filtre médian remplace chaque pixel par la médiane de son voisinage dans l'image d'entrée.

La médiane résiste aux valeurs extrêmes isolées et préserve souvent mieux les transitions nettes que la moyenne. Elle est particulièrement utile contre le bruit impulsionnel, mais n'est pas systématiquement meilleure pour toute image et tout modèle de bruit. De grandes fenêtres peuvent effacer les objets fins et les petits détails. La figure 2.6 illustre ce compromis.

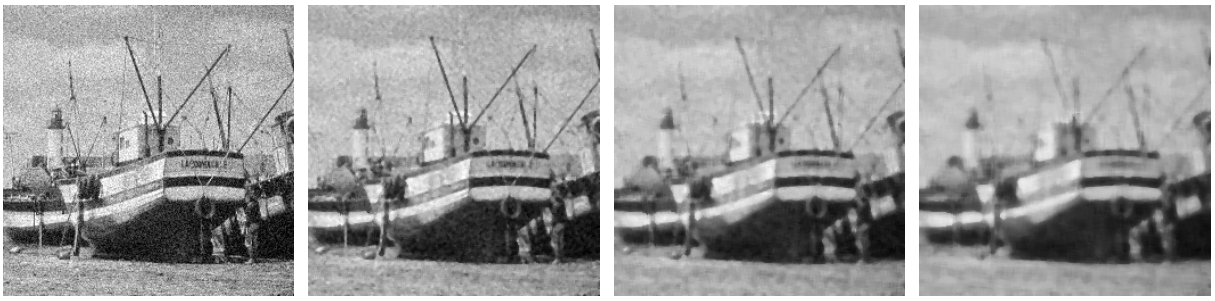


Image bruitée | Médiane 3×3 | Médiane 5×5 | Médiane 7×7

Figure 2.6. Filtrage médian avec une fenêtre de taille croissante.

2.4 Détecter les contours

Les contours sont des zones où les valeurs des pixels varient rapidement, par exemple à la frontière entre un objet sombre et un fond clair. Un détecteur simple compare les voisins situés de part et d'autre d'un pixel.

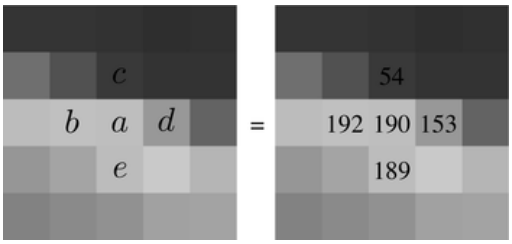


Figure 2.7. Le pixel central a et ses voisins gauche, supérieur, droit et inférieur b, c, d, e.

Avec la disposition de la figure 2.7, on définit

$$\ell = \sqrt{(b-d)^2 + (c-e)^2}.$$

Pour un pas spatial égal à un pixel, $\ell/2$ est la norme du gradient estimé par différences centrées. Les valeurs de la figure donnent

$$\ell = \sqrt{(192-153)^2 + (54-189)^2} = \sqrt{19\,746} \simeq 140{,}52.$$

On a $\ell = 0$ exactement lorsque $b = d$ et $c = e$. Cela n'implique pas que les cinq valeurs soient égales : le détecteur peut manquer certaines oscillations fines. Une grande valeur indique une variation locale importante, due à un contour, à une texture ou au bruit.

Pour l'affichage, on tronque la valeur à $\min(\ell, 255)$ puis on l'arrondit. Le minimum est nécessaire car ℓ peut dépasser 255. La figure 2.8 affiche les fortes réponses en blanc. Débruiter avant de dériver peut être utile, car les différences amplifient aussi le bruit.

2.5 Images en couleur

2.5.1 Canaux RVB et synthèse additive

Une image RVB comporte trois canaux, représentant le rouge, le vert et le bleu. Ce sont des tableaux distincts, mais ils ne sont généralement *pas statistiquement indépendants*. Dans une représentation sur huit bits par canal, un pixel est un triplet $(r, v, b) \in \{0, \ldots, 255\}^3$ et demande 24 bits avant compression.



Image d'origine Réponse de contour ℓ

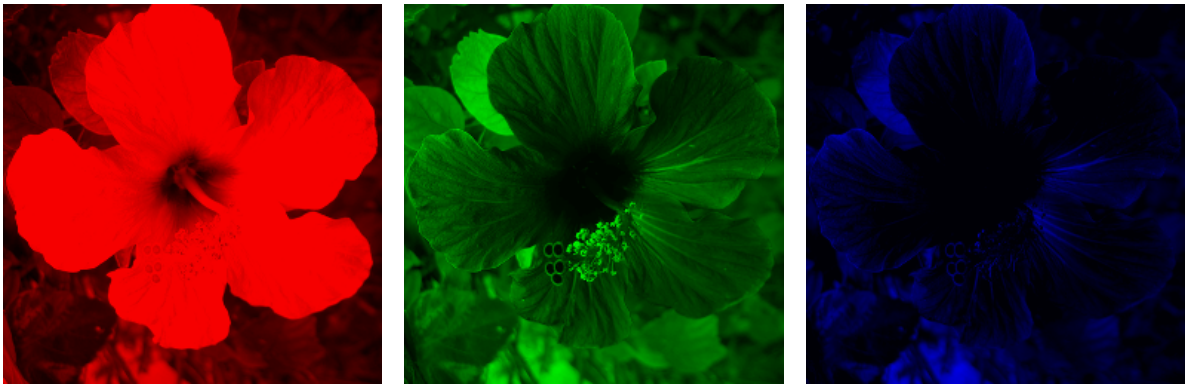
Figure 2.8. Détection des contours par différences entre voisins opposés.

Les valeurs $(255, 0, 0)$, $(0, 255, 0)$ et $(0, 0, 255)$ donnent respectivement du rouge, du vert et du bleu. Le mélange de lumières émises obéit à la synthèse *additive* : rouge et vert, $(255, 255, 0)$, donnent du jaune ; rouge et bleu, $(255, 0, 255)$, donnent du magenta. Le blanc correspond à $(255, 255, 255)$ et le noir à $(0, 0, 0)$.



Image en couleur Moyenne des canaux

Figure 2.9. Une image en couleur et une représentation en niveaux de gris.



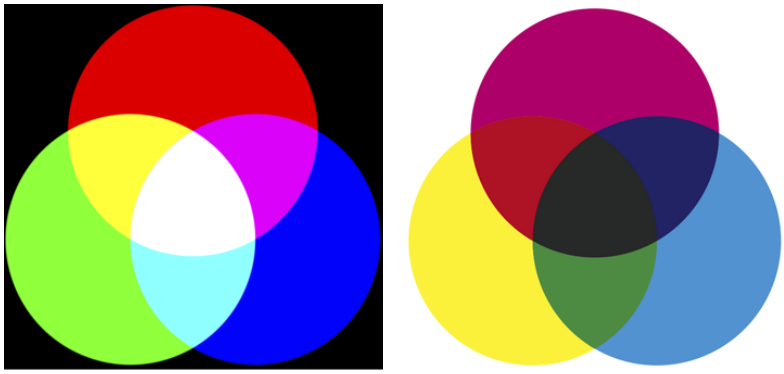
Canal rouge Canal vert Canal bleu

Figure 2.10. Les trois canaux RVB affichés dans leurs couleurs respectives.

2.5.2 Canaux CMJ et synthèse soustractive

Une représentation complémentaire idéalisée utilise le cyan, le magenta et le jaune :

$$c = 255 - r, \qquad m = 255 - v, \qquad j = 255 - b.$$



Synthèse additive Synthèse soustractive

Figure 2.11. Modèles idéalisés des synthèses additive et soustractive.

Par exemple, le bleu $(r, v, b) = (0, 0, 255)$ correspond à $(c, m, j) = (255, 255, 0)$. Ces variables décrivent l'action *soustractive* de pigments idéaux sur la lumière blanche : le cyan absorbe le rouge, le magenta absorbe le vert et le jaune absorbe le bleu.



Canal cyan Canal magenta Canal jaune

Figure 2.12. Les canaux complémentaires CMJ, représentés sur fond blanc.

Les imprimantes réelles utilisent des conversions dépendant du matériel, généralement avec un canal noir supplémentaire (CMJN). Un écran émet toujours de la lumière RVB : les données CMJ doivent donc être converties pour y être affichées. Ni les primaires d'un écran donné ni les encres d'une imprimante donnée ne reproduisent toutes les couleurs visibles.

2.6 Modifier la luminosité et le contraste

2.6.1 Une intensité en niveaux de gris

Une manière simple de calculer une image en niveaux de gris consiste à moyenner les trois canaux :

$$a = \frac{r + v + b}{3}.$$

C'est l'intensité employée dans les exemples élémentaires de ce chapitre. Ce n'est pas la *luminance* physique, qui pondère des composantes lumineuses linéaires selon la sensibilité visuelle. La moyenne arithmétique de valeurs RVB codées ne mesure donc pas de manière calibrée la lumière émise. [2]

2.6.2 Transformations ponctuelles en niveaux de gris

Remplacer a par $255 - a$ échange le noir et le blanc et produit un négatif. Pour assombrir une image, on utilise $a \mapsto a^2/255$; pour l'éclaircir, on utilise $a \mapsto \sqrt{255a}$. Ces deux applications conservent l'intervalle $[0, 255]$.

2. Pour la définition de la luminance relative en sRGB, voir https://www.w3.org/TR/WCAG22/#dfn-relative-luminance.



Négatif | Carré : plus sombre | Racine : plus clair

Figure 2.13. Trois transformations ponctuelles de l'intensité en niveaux de gris.

Les fonctions carré et racine carrée sont réciproques sur l'intervalle réel. Après arrondi à des valeurs de pixels entières, elles ne se compensent plus nécessairement de façon exacte. Elles se généralisent par la transformation de puissance

$$\widetilde{a} = 255\left(\frac{a}{255}\right)^{\gamma}, \qquad \gamma > 0,$$

avec $\widetilde{a} = 0$ pour $a = 0$. Pour $0 < a < 255$, l'image est éclaircie lorsque $\gamma < 1$ et assombrie lorsque $\gamma > 1$. Les cas $\gamma = 1/2$ et $\gamma = 2$ correspondent respectivement à la racine carrée et au carré ; $\gamma = 1$ laisse les valeurs inchangées.

2.6.3 Une transformation en couleur

Appliquer des modifications sans lien aux trois canaux peut altérer fortement les couleurs. Une solution élémentaire consiste à modifier l'intensité moyenne a tout en conservant le résidu $(r - a, v - a, b - a)$. On pose

$$\begin{aligned} \widetilde{r} &= \mathrm{clip}(r + \widetilde{a} - a), \\ \widetilde{v} &= \mathrm{clip}(v + \widetilde{a} - a), \\ \widetilde{b} &= \mathrm{clip}(b + \widetilde{a} - a), \end{aligned} \qquad \mathrm{clip}(u) = \max(0, \min(255, u)).$$

Sans troncature, la nouvelle moyenne vaut $\widetilde{a}$ et les différences entre canaux sont conservées. La troncature peut modifier ces deux propriétés ; cette construction ne garantit donc pas une conservation exacte de la teinte ou de la saturation perçues. Les valeurs ne sont arrondies qu'à la fin du calcul.



$\gamma = 0{,}5$ | 0,75 | 1 | 1,5 | 2 | 3

Figure 2.14. Modifier une image en couleur par une transformation de puissance de son intensité moyenne.

2.7 Images et matrices

2.7.1 Transposition et rotation

Écrivons une image comme une matrice $A = (a_{i,j}) \in \mathbb{R}^{n \times p}$. Les lignes sont indexées de haut en bas par $1 \leqslant i \leqslant n$, les colonnes de gauche à droite par $1 \leqslant j \leqslant p$. La transposée $B = A^{\mathsf{T}} \in \mathbb{R}^{p \times n}$ vérifie

$$b_{j,i} = a_{i,j}.$$

Elle échange les deux directions de coordonnées et réfléchit l'image par rapport à la direction diagonale. Un quart de tour dans le sens des aiguilles d'une montre produit une autre matrice C de taille $p \times n$, définie par

$$c_{j,i} = a_{n-i+1,j}, \qquad 1 \leqslant j \leqslant p, \quad 1 \leqslant i \leqslant n.$$

Autrement dit, le pixel initial (i, j) se déplace en $(j, n - i + 1)$. Ces opérations permutent les valeurs des pixels sans perdre d'information. Pour une image en couleur, on applique la même permutation à tous les canaux.



A A^{T} Rotation horaire

Figure 2.15. Transposition et rotation d'un quart de tour.

2.7.2 Fondu entre deux images

Pour deux images A et B de mêmes dimensions et de même format de canaux, on définit

$$C_t = (1 - t)A + tB, \qquad 0 \leqslant t \leqslant 1.$$

Cette formule s'applique à chaque pixel et à chaque canal. Aux extrémités, $C_0 = A$ et $C_1 = B$; les valeurs intermédiaires produisent un *fondu*. Comme les poids sont positifs et de somme un, les valeurs restent dans $[0, 255]$ avant arrondi. Cette opération mélange les couleurs à positions fixes ; elle ne déplace pas les objets d'une position à une autre.



$t = 0$ 0,2 0,4 0,6 0,8 1

Figure 2.16. Interpolation linéaire de l'image A vers l'image B.

Pour aller plus loin. Les Numerical Tours [34] proposent des expériences reproductibles. Ce chapitre reprend en partie l'article de l'auteur *Le traitement numérique des images*, paru dans *Images des mathématiques*.

CHAPITRE 3

Parcimonie, problèmes inverses et acquisition comprimée

Comment décrire un signal complexe à partir d'un petit nombre de mesures ? Les représentations parcimonieuses relient la compression à la reconstruction de données incomplètes ou bruitées. Ce chapitre développe ce lien, explique le rôle de l'optimisation convexe et précise les hypothèses qui permettent de garantir une reconstruction par acquisition comprimée.

3.1 Du signal continu aux échantillons

Un dispositif d'acquisition transforme un signal continu $\tilde{f}$ en un vecteur fini $f = (f_q)_{q=1}^{Q} \in \mathbb{R}^Q$. Pour un son, ces valeurs correspondent à des mesures effectuées à des instants successifs. Pour une image en niveaux de gris, elles représentent la lumière reçue par de petites surfaces du capteur. Une mesure de pixel idéalisée s'écrit

$$f_q = \int_{c_q} \tilde{f}(s)\,\mathrm{d}s,$$

où c_q est la surface de l'élément sensible. Diviser par son aire donne une intensité moyenne ; le temps d'exposition et l'étalonnage introduisent d'autres facteurs d'échelle. Les signaux en couleur ou multicanaux se décrivent en enregistrant plusieurs valeurs à chaque position.

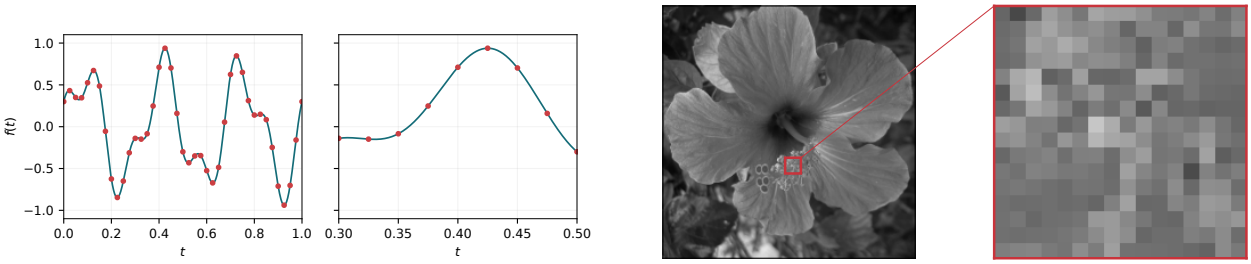


FIGURE 3.1. Échantillonnage d'un signal unidimensionnel et d'une image. Les points indiquent les échantillons temporels ; les cadres rouges repèrent la région de l'image agrandie à droite.

L'échantillonnage préserve l'information sous des hypothèses appropriées. Supposons par exemple que $\tilde{f} \in L^2(\mathbb{R})$ ait une transformée de Fourier à support dans $[-B, B]$, la fréquence étant mesurée en cycles par unité de temps. Si le pas vérifie $0 < T < 1/(2B)$, le théorème d'échantillonnage [42] donne

$$\tilde{f}(t) = \sum_{k\in\mathbb{Z}} \tilde{f}(kT)\,\operatorname{sinc}\!\left(\frac{t}{T} - k\right), \qquad \operatorname{sinc}(u) = \begin{cases} \sin(\pi u)/(\pi u), & u \neq 0, \\ 1, & u = 0. \end{cases}$$

Pour le représentant continu de cette fonction à bande limitée, la série converge dans L^2 et uniformément. Le théorème utilise les échantillons sur tout l'axe temporel. Un enregistrement fini, la quantification et le bruit de mesure introduisent des erreurs supplémentaires. Les fréquences dépassant la bande d'acquisition doivent être atténuées avant l'échantillonnage pour éviter le repliement spectral ; augmenter ensuite la précision numérique ne supprime pas ce repliement.

L'échantillonnage concerne la représentation d'un signal continu par des mesures, tandis que le codage sans perte concerne la représentation binaire de ces mesures. La parcimonie fournit une autre hypothèse utile sur le signal et conduit à une autre stratégie d'acquisition.

3.2 Approximation non linéaire et compression

3.2.1 Représentations parcimonieuses

Un dictionnaire $\Psi = (\psi_n)_{n=1}^N$ est une famille de vecteurs $\psi_n \in \mathbb{R}^Q$. On cherche une approximation

$$f \approx \Psi x = \sum_{n=1}^{N} x_n \psi_n$$

comportant peu de coefficients non nuls. Les sinusoïdes, les cosinus locaux et les ondelettes sont des exemples d'atomes. Le codage par transformée utilise ces représentations pour concentrer une grande partie de l'énergie du signal dans un petit nombre de coefficients.

Pour un budget entier $0 \leq M \leq N$, une meilleure approximation à M termes résout

$$x^\star \in \arg\min_{x \in \mathbb{R}^N,\ \|x\|_0 \leq M} \|f - \Psi x\|_2^2, \qquad \|x\|_0 = \#\{n : x_n \neq 0\}. \tag{3.1}$$

On note ici $\|v\|_2^2 = \sum_n |v_n|^2$. La notation $\|x\|_0$ est conventionnelle : ce comptage n'est pas une norme, car multiplier un vecteur non nul par un scalaire non nul ne change pas le nombre de ses composantes non nulles. Pour un dictionnaire général, l'approximation parcimonieuse est un problème NP-difficile [33]. Tester tous les supports possibles est généralement prohibitif ; cette difficulté dans le pire cas n'exclut pas des algorithmes efficaces pour certains dictionnaires ou certaines données.

3.2.2 Le cas orthonormal

Supposons $N = Q$ et Ψ base orthonormale, c'est-à-dire $\langle \psi_n, \psi_m \rangle = \delta_{nm}$. En posant $a_n = \langle f, \psi_n \rangle$, l'identité de Parseval donne

$$f = \sum_{n=1}^{N} a_n \psi_n, \qquad \|f - \Psi x\|_2^2 = \sum_{n=1}^{N} |a_n - x_n|^2. \tag{3.2}$$

Une fois le support choisi, les meilleurs coefficients sont les a_n correspondants. Pour minimiser l'énergie éliminée, on conserve les M plus grandes amplitudes. Plus précisément, choisissons un ordre $|a_{n_1}| \geq \cdots \geq |a_{n_N}|$. Alors

$$x_n^\star = \begin{cases} a_n, & n \in \{n_1, \ldots, n_M\}, \\ 0, & \text{sinon.} \end{cases} \tag{3.3}$$

Des égalités d'amplitude au niveau du seuil peuvent donner plusieurs minimiseurs. S'il y a moins de M coefficients non nuls, l'approximation utilise moins de M termes.



f $\quad$ $M = N/4$ $\quad$ $M = N/8$ $\quad$ $M = N/16$

Figure 3.2. Meilleures approximations à M termes d'une image de fleur de $N = 256^2$ pixels. La transformée orthonormale utilise des ondelettes de Daubechies à quatre moments nuls, cinq niveaux et des conditions périodiques au bord. Les intensités affichées sont tronquées à $[0, 1]$.

La reconstruction $x \mapsto \Psi x$ est linéaire, mais la sélection des plus grands coefficients dépend des données. L'approximation $f \mapsto \Psi x^\star$ est donc non linéaire. Pour de nombreux signaux réguliers par morceaux, les ondelettes donnent de petites erreurs avec relativement peu de coefficients ; la régularité du signal contrôle la décroissance de cette erreur [29, chapitre 9]. Les textures et les oscillations fines peuvent demander bien davantage de termes.

Le rapport M/N compte les coefficients conservés ; ce n'est pas un débit binaire. Un compresseur complet doit également quantifier leurs valeurs, décrire leurs positions et coder les symboles obtenus. Le codage entropique, introduit au chapitre consacré à la théorie de l'information, complète cette étape. Les transformées en ondelettes disposent en outre d'algorithmes rapides dont le coût est proportionnel au nombre d'échantillons [29, chapitre 7].

3.3 Problèmes inverses et régularisation

3.3.1 Le modèle d'observation

Un problème inverse consiste à retrouver un signal inconnu $f \in \mathbb{R}^Q$ à partir de mesures

$$y = \Phi f + w \in \mathbb{R}^P. \tag{3.4}$$

L'opérateur linéaire connu $\Phi \in \mathbb{R}^{P\times Q}$ décrit l'acquisition, et w représente les erreurs inconnues. On applique généralement Φ par un algorithme plutôt que de stocker cette matrice potentiellement immense.



f Φf : flou Φf : masque

Figure 3.3. Deux opérateurs d'observation sans bruit appliqués à la même image de fleur : un flou gaussien et un masque conservant environ 10% des pixels. Les valeurs manquantes sont affichées en noir.

Plusieurs problèmes classiques entrent dans ce cadre :

- Débruitage : $\Phi = \mathrm{Id}$ et $P = Q$.
- Déconvolution : $\Phi f = \varphi \star f$, où le filtre φ décrit le flou et où les conditions au bord sont précisées.
- Données manquantes : Φ sélectionne les coordonnées observées. On peut aussi conserver le tableau complet et utiliser le masque diagonal $\Phi = \mathrm{diag}(\mu_q)$, avec $\mu_q \in \{0, 1\}$.
- Tomographie : Φ modélise des intégrales de l'objet le long de droites, comme une transformée de Radon discrétisée.

Si Φ a un noyau non trivial, les seules mesures ne déterminent pas f. Même lorsqu'une matrice carrée Φ est inversible, ses petites valeurs singulières peuvent amplifier les erreurs dans $\Phi^{-1}y = f + \Phi^{-1}w$. La régularisation apporte une information supplémentaire pour sélectionner une reconstruction utile et stable.

3.3.2 Régularisation parcimonieuse

L'hypothèse $f \approx \Psi x$ avec peu de coefficients actifs conduit à

$$x^\star \in \arg\min_{\|x\|_0 \le M} \|y - \Phi\Psi x\|_2^2, \qquad f^\star = \Psi x^\star. \tag{3.5}$$

Pour le débruitage dans une base orthonormale, on obtient un seuillage dur des coefficients du *signal observé* y. Il fournit le meilleur ajustement parcimonieux à y pour le budget choisi ; il ne garantit pas, à lui seul, la plus petite erreur par rapport au signal propre inconnu f. Pour un opérateur d'acquisition général, $\Phi\Psi$ n'est pas orthogonal et la formule de seuillage ne résout plus le problème.

Les algorithmes gloutons et la relaxation convexe proposent deux façons d'aborder cette difficulté [30, 45]. Pour motiver la relaxation convexe, considérons

$$\|x\|_\alpha^\alpha = \sum_{n=1}^{N} |x_n|^\alpha, \qquad \alpha > 0.$$

Pour chaque vecteur x fixé, cette quantité tend vers $\|x\|_0$ lorsque $\alpha \downarrow 0$. Les ensembles $B_\alpha = \{x : \sum_n |x_n|^\alpha \le 1\}$ sont contenus dans $[-1,1]^N$ et convergent, pour la distance de Hausdorff, vers

$$B_0 = \{x \in [-1,1]^N : \|x\|_0 \le 1\}.$$

Cet ensemble limite est constitué de segments portés par les axes de coordonnées, y compris l'origine. L'ensemble $\{x : \|x\|_0 \le 1\}$ sans restriction au cube contient, lui, les axes entiers et n'est pas borné.

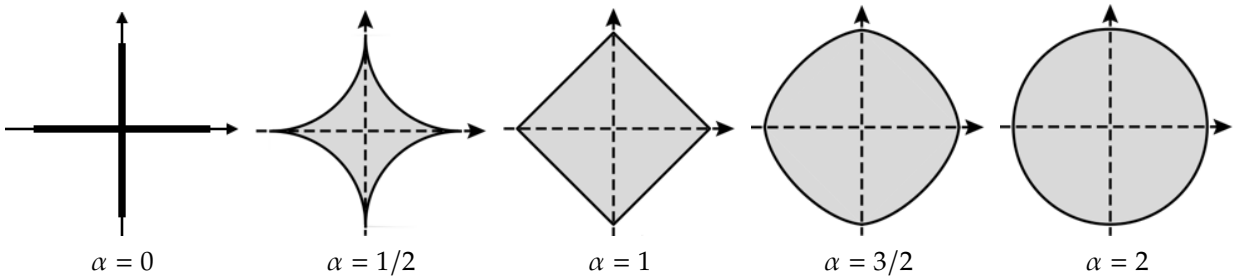


Figure 3.4. Les ensembles B_α en dimension deux. Le premier panneau représente l'ensemble limite borné B_0.

Dans cette famille, $\|\cdot\|_\alpha$ est une norme et B_α est convexe exactement lorsque $\alpha \ge 1$. Le choix $\alpha = 1$ associe la convexité à une géométrie qui favorise les coefficients nuls. On obtient ainsi le problème du Lasso sous contrainte

$$x^\star \in \arg\min_{\|x\|_1 \le \tau} \frac{1}{2}\|Ax - y\|_2^2, \qquad A = \Phi\Psi. \tag{3.6}$$

Augmenter τ permet de mieux ajuster les mesures, mais peut aussi conduire à reproduire leurs erreurs. Ce paramètre se choisit en fonction du bruit, de l'opérateur et du modèle de signal. Faire tendre τ vers l'infini supprime la régularisation et laisse généralement plusieurs reconstructions possibles lorsque $P < N$.

Deux formulations proches sont utiles :

$$\min_x \frac{1}{2}\|Ax - y\|_2^2 + \lambda\|x\|_1, \qquad \lambda > 0, \tag{3.7}$$

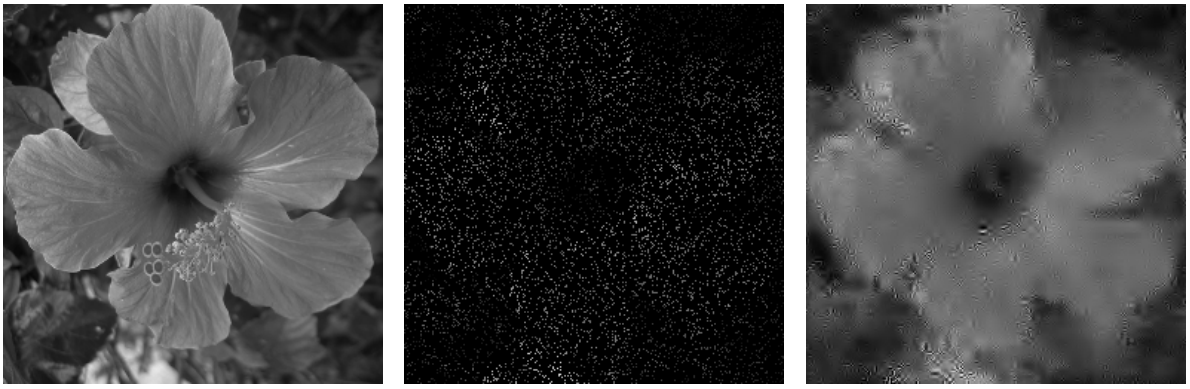
$$\min_x \|x\|_1 \quad \text{sous la contrainte } \|Ax - y\|_2 \le \varepsilon, \qquad \varepsilon \ge 0. \tag{3.8}$$

La formulation pénalisée équilibre fidélité aux données et parcimonie. La seconde, appelée *basis pursuit denoising*, est commode lorsqu'une borne d'erreur ε est connue. Pour $\varepsilon = 0$, elle devient le problème de *basis pursuit*, $\min_{Ax=y}\|x\|_1$. Ces formulations sont reliées par des multiplicateurs de Lagrange appropriés ; leurs paramètres ne sont pas des valeurs numériques interchangeables. Leurs premiers développements concernent notamment l'inversion sismique, la décomposition par *basis pursuit* et le Lasso [10, 40, 44].

La convexité rend possible l'optimisation globale, mais la grande dimension et la non-différentiabilité de $\|x\|_1$ demandent des algorithmes adaptés. Les méthodes proximales exploitent le seuillage doux explicite associé à cette pénalité [11]. Un solveur numérique atteint une précision prescrite, plutôt qu'une solution réelle exacte après un petit nombre d'opérations garanti.

3.3.3 Le rôle de la géométrie

En dimension deux, la boule ℓ^1 possède des sommets sur les axes de coordonnées. Un ensemble de niveau du terme d'attache aux données peut rencontrer la boule pour la première fois en l'un de ces sommets, ce qui produit un coefficient nul. Dans la même configuration, la boule ronde ℓ^2 donne généralement deux coefficients non nuls (figure 3.6). Cette géométrie explique une tendance, sans garantir le résultat pour tout opérateur et toute donnée : un contact le long d'une face peut donner plusieurs minimiseurs, et une solution ℓ^1 n'est pas nécessairement assez parcimonieuse pour retrouver le signal recherché.



f y $f^\star = \Psi x^\star$

Figure 3.5. Reconstruction à partir de 6 554 pixels observés sur 256^2, soit environ 10%. Le problème pénalisé (3.7) est résolu numériquement avec $\lambda = 0{,}005$, des intensités dans $[0,1]$ et les mêmes ondelettes orthonormales qu'à la figure 3.2. Les erreurs de texture restantes illustrent les limites du modèle.

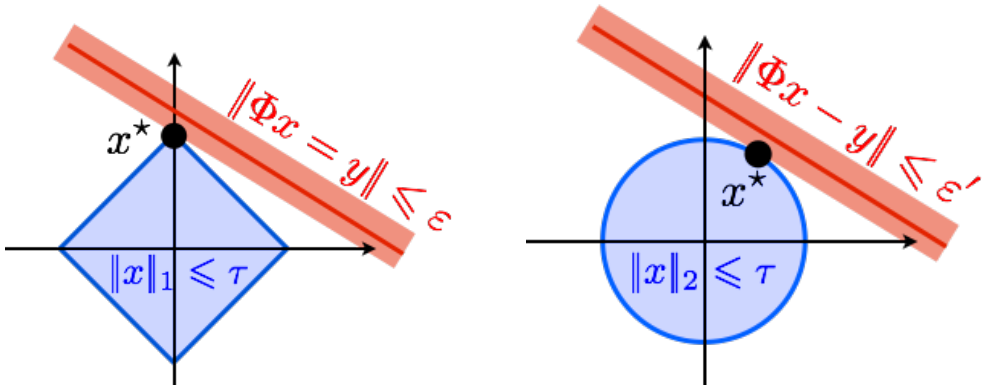


Figure 3.6. Ajustement aux données sous une contrainte ℓ^1 (à gauche) ou ℓ^2 (à droite). À partir de $\varepsilon = 0$, on élargit la bande $\{x : \|Ax - y\|_2 \leq \varepsilon\}$ jusqu'à son premier contact avec la boule de contrainte. Le point de contact minimise le résidu sur cette boule.

La reconstruction dépend de l'action de l'opérateur de mesure sur les vecteurs parcimonieux. Augmenter la dimension ambiante ne suffit pas à l'améliorer. Des conditions précises existent pour de nombreux opérateurs aléatoires et structurés ; la convolution et l'acquisition de Fourier demandent des hypothèses adaptées à leur géométrie [1, 8, 9, 18].

3.4 Acquisition comprimée

3.4.1 Mesurer des combinaisons de pixels

Le codage par transformée commence par acquérir Q échantillons, puis en conserve une représentation beaucoup plus petite. L'acquisition comprimée cherche à mesurer directement $P \ll Q$ combinaisons linéaires bien choisies et à reconstruire un signal parcimonieux, ou approximativement parcimonieux, dans une base connue.

La caméra à pixel unique développée à l'université Rice illustre cette idée [17]. Une grille de Q micromiroirs dirige certaines portions de la lumière vers un détecteur unique ; les autres miroirs les dirigent ailleurs. Pour chaque configuration p, le détecteur mesure la somme

$$y_p = \sum_{q=1}^{Q} \Phi_{p,q} \int_{c_q} \tilde{f}(s)\,\mathrm{d}s + w_p = \sum_{q=1}^{Q} \Phi_{p,q} f_q + w_p, \qquad \Phi_{p,q} \in \{0,1\}.$$

Les miroirs n'enregistrent pas les pixels individuels. Changer leur configuration produit directement le vecteur $y \in \mathbb{R}^P$. La scène doit rester suffisamment stable pendant ces mesures. Des configurations complémentaires, ou une mesure étalonnée de la lumière totale, permettent de convertir les mesures $0/1$ en mesures signées centrées.

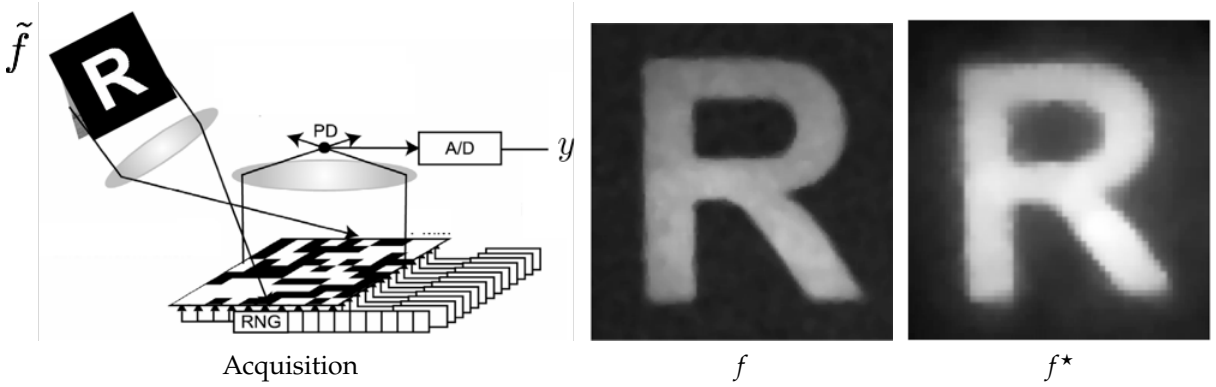


Figure 3.7. Acquisition à pixel unique et exemple de reconstruction original, repris des documents associés à [17]. Le facteur annoncé de réduction du nombre de mesures est $Q/P = 6$, donc $P/Q \approx 1/6$.

3.4.2 Une garantie de reconstruction

Voici une forme classique de la garantie d'acquisition comprimée [7, 16]. Supposons que Ψ soit une base orthonormale de $\mathbb{R}^N$, que $f = \Psi x_0$ et que $A = \Phi\Psi$ ait des entrées gaussiennes indépendantes, de moyenne nulle et de variance $1/P$. On peut, de façon équivalente, choisir Φ suivant cette loi gaussienne, qui est invariante par multiplication par une matrice orthogonale fixée.

Il existe des constantes universelles positives C, c, C_0, C_1 telles que, pour $1 \le M \le N$ et

$$P \ge CM \log(eN/M), \tag{3.9}$$

avec une probabilité au moins égale à $1 - 2e^{-cP}$ sur le choix de A, la borne suivante soit vraie simultanément pour tous les x_0 et toutes les erreurs vérifiant $\|w\|_2 \le \varepsilon$. Toute solution $x^\star$ de (3.8), avec $y = Ax_0 + w$, satisfait

$$\|x^\star - x_0\|_2 \le C_0 \frac{\sigma_M(x_0)_1}{\sqrt{M}} + C_1\varepsilon, \qquad \sigma_M(x_0)_1 = \min_{\|z\|_0 \le M} \|x_0 - z\|_1. \tag{3.10}$$

Le logarithme de (3.9) est naturel ; changer sa base modifie seulement la constante. Le régime comprimé intéressant est celui où le nombre requis P reste inférieur à N.

La quantité $\sigma_M(x_0)_1$ est la somme des amplitudes des coefficients éliminés après conservation des M plus grands. Si x_0 est exactement M-parcimonieux et si l'on choisit $\varepsilon = 0$ dans le problème sans bruit (donc $w = 0$), les deux termes de (3.10) s'annulent : le *basis pursuit* retrouve exactement x_0. Pour les signaux approximativement parcimonieux et les mesures bruitées, ces termes séparent l'erreur d'approximation de l'erreur de mesure. L'orthonormalité donne la même erreur euclidienne sur le signal reconstruit : $\|\Psi x^\star - f\|_2 = \|x^\star - x_0\|_2$.

Une démonstration de ce théorème passe par la propriété d'isométrie restreinte : A préserve approximativement les longueurs euclidiennes des vecteurs portés par des ensembles suffisamment petits. Les matrices aléatoires vérifient cette propriété lorsque le nombre de mesures est de l'ordre indiqué dans (3.9). Des modèles sous-gaussiens centrés appropriés admettent aussi des garanties, avec des constantes dépendant de leur loi. Pour une matrice brute à entrées $0/1$, il faut tenir compte du centrage et de la normalisation ; ce n'est pas le modèle gaussien énoncé ci-dessus.

Le théorème porte sur un nombre de mesures réelles, et non sur un nombre de bits. Leur précision, leur quantification et leur stockage restent à prendre en compte. L'algorithme de reconstruction doit connaître la matrice de mesure, par exemple grâce à une graine aléatoire partagée. Garder cette matrice secrète ne constitue pas, à lui seul, une garantie de sécurité cryptographique. De même, les économies de temps ou d'énergie dépendent du dispositif physique et du coût de reconstruction.

Les acquisitions structurées, notamment en imagerie par résonance magnétique, imposent des contraintes supplémentaires : on ne peut généralement pas choisir des mesures gaussiennes indépendantes arbitraires. Les trajectoires d'échantillonnage et les modèles de parcimonie doivent donc être étudiés ensemble [1, 9]. Le principe directeur reste de concevoir conjointement l'acquisition et la reconstruction, en s'appuyant sur un modèle adapté aux données.

Pour aller plus loin. Les liens entre approximation, problèmes inverses et optimisation dépassent la parcimonie. D'autres pénalités décrivent la régularité, le faible rang ou la structure géométrique. L'article de synthèse [45] développe ces idées, et les Numerical Tours [34] proposent des expériences numériques.

CHAPITRE 4

Transport optimal

Comparer deux distributions demande de préciser le coût du déplacement de la masse entre les positions. Le transport optimal traduit cette idée en un problème d'optimisation. À partir d'un petit problème d'affectation, nous introduisons les plans de transport, expliquons quand ils correspondent à des applications, puis utilisons les distances de transport pour comparer des couleurs, des formes et des textes.

4.1 Le problème de Monge

En 1781, Gaspard Monge formule le problème du déplacement de terre d'un lieu à un autre à moindre coût [32]. Son coût est la distance parcourue par chaque unité de masse. D'autres applications suggèrent d'autres coûts, comme le temps de trajet ou le carré de la distance euclidienne.

4.1.1 Un problème d'affectation

Considérons six boulangeries et six cafés à Paris. Chaque boulangerie fournit un lot de croissants, et chaque café demande un lot. Dans le modèle d'affectation, chaque lot va à un seul café, et chaque café reçoit un seul lot. Les deux exigences sont nécessaires : donner simplement une destination à chaque boulangerie permettrait à plusieurs d'entre elles de choisir le même café.

Notons C_{ij} le temps de trajet de la boulangerie i au café j. Dans l'exemple de la figure 4.1, on a $C_{34} = 10$. Une affectation admissible est une permutation σ de $\{1, \dots, 6\}$. Par exemple,

$$(\sigma(1), \dots, \sigma(6)) = (5, 2, 6, 1, 3, 4). \tag{4.1}$$

Son coût total est

$$\text{Coût}(\sigma) = \sum_{i=1}^{6} C_{i,\sigma(i)} = 10 + 7 + 15 + 10 + 14 + 9 = 65. \tag{4.2}$$

On additionne ici les six temps de trajet. Ce modèle ne décrit ni la tournée d'un véhicule unique, ni la durée totale de livraisons effectuées en parallèle.

Pour n lots identiques de chaque côté, le problème discret de Monge s'écrit

$$\min_{\sigma \in \Sigma_n} \sum_{i=1}^{n} C_{i,\sigma(i)}, \tag{4.3}$$

où Σ_n est l'ensemble des permutations. Dans notre exemple, la valeur optimale est 64, atteinte par $(5, 6, 2, 1, 3, 4)$. L'examen exhaustif des $6! = 720$ possibilités permet de le vérifier. Il y a $n!$ affectations en général, et $70! \approx 1{,}198 \times 10^{100}$: l'énumération devient donc vite impraticable.

4.1.2 Trier en dimension un

Supposons que les positions sources $x_1 \le \cdots \le x_n$ et cibles $y_1 \le \cdots \le y_n$ soient sur une droite, avec le coût $C_{ij} = |x_i - y_j|$. Apparier les points dans l'ordre croissant est optimal. L'inégalité essentielle est

$$|x - y| + |x' - y'| \le |x - y'| + |x' - y| \quad \text{si } x \le x' \text{ et } y \le y'.$$

On la vérifie en ordonnant les quatre points sur la droite. Lorsqu'une permutation contient une inversion, échanger les deux destinations n'augmente pas le coût. Éliminer successivement les inversions donne l'appariement trié. Une égalité est possible : cette affectation optimale n'est donc pas nécessairement unique.

Le coût du calcul est ainsi celui du tri des deux listes. Le tri par insertion effectue au plus $n(n-1)/2$ comparaisons par liste, soit 2 415 pour $n = 70$. Le tri fusion ramène ce coût à l'ordre de $n \log n$. Il s'agit du

c_{ij}	y_1	y_2	y_3	y_4	y_5	y_6
x_1	12	10	31	27	10	30
x_2	22	7	25	15	11	14
x_3	19	7	19	10	15	15
x_4	10	6	21	19	14	24
x_5	15	23	14	24	31	34
x_6	35	26	16	9	34	15

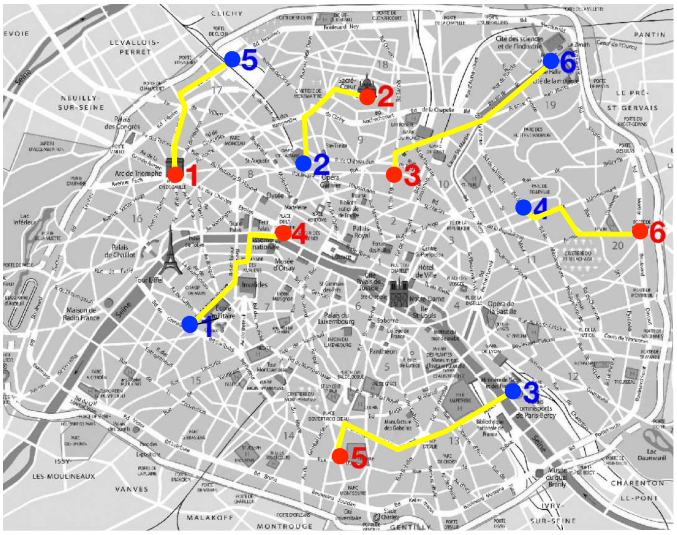


Figure 4.1. Matrice de coût et affectation des six boulangeries (rouge) aux six cafés (bleu). Les entrées surlignées donnent un temps de trajet cumulé de 65 minutes.

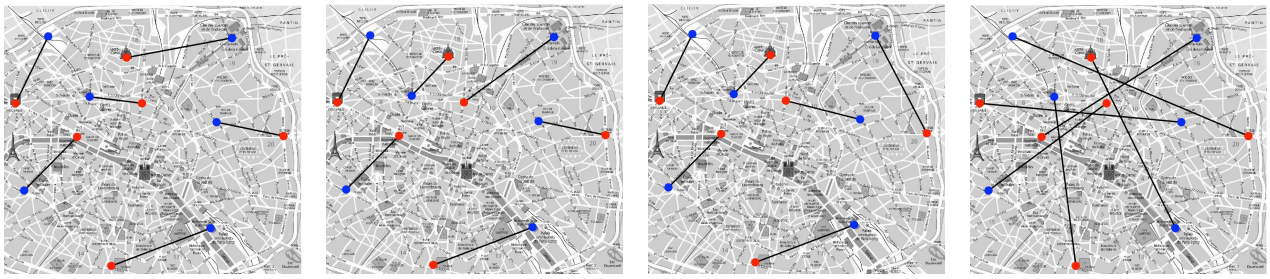


Figure 4.2. Quatre affectations de coûts 64, 65, 66 et 152, de gauche à droite.

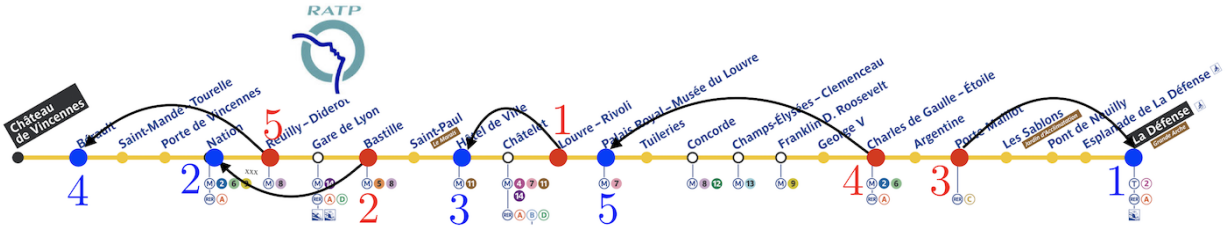


Figure 4.3. Appariement trié le long d'une ligne de métro, considérée comme un axe unidimensionnel. Pour les indices affichés, $\sigma : (1, 2, 3, 4, 5) \mapsto (3, 2, 1, 5, 4)$.

nombre de comparaisons pour le tri, et non d'une estimation de toutes les opérations d'une application de transport.

En dimension deux, aucun ordre unique ne résout le problème général. Pour le coût euclidien, deux segments d'un appariement optimal ne peuvent toutefois pas se croiser transversalement en leurs intérieurs. Si les segments $[x, y]$ et $[x', y']$ se rencontrent en z, l'inégalité triangulaire donne

$$|x - y'| + |x' - y| \leq |x - z| + |z - y'| + |x' - z| + |z - y| = |x - y| + |x' - y'|.$$

Pour un croisement transverse, l'inégalité est stricte : échanger les destinations diminue le coût. Des recouvrements colinéaires ou des extrémités communes sont des cas dégénérés qui ne donnent pas nécessairement d'amélioration stricte. Cette observation ne suffit pas à calculer un optimum : de nombreux appariements sans croisement peuvent avoir des coûts différents.

4.2 La relaxation de Kantorovitch

La formulation de Kantorovitch autorise le partage de la masse entre plusieurs destinations [24]. Elle remplace les choix discrets par des quantités positives ou nulles soumises à des contraintes linéaires de

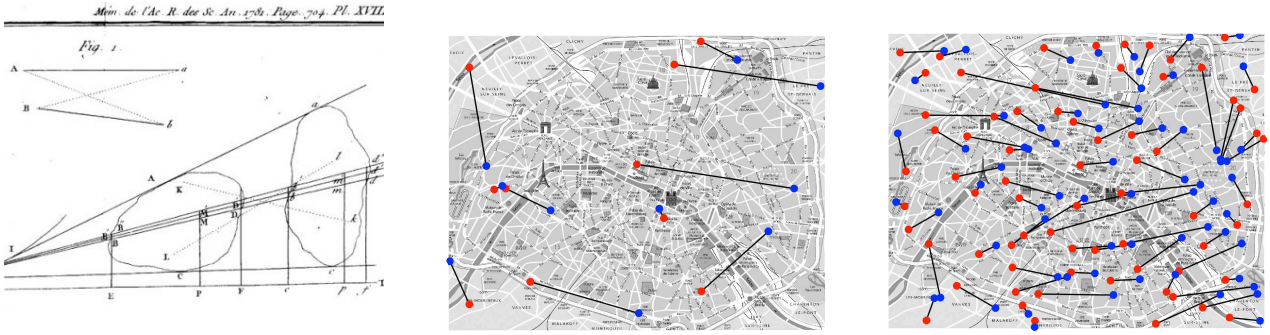


FIGURE 4.4. Une illustration de l'article original de Monge [32] et deux appariements euclidiens optimaux dans le plan.

conservation, faisant du transport optimal un problème de programmation linéaire.

4.2.1 Des permutations aux matrices

On représente une permutation par $P_{ij} = 1$ si $j = \sigma(i)$ et $P_{ij} = 0$ sinon. Par exemple, les permutations $(1,2,3)$, $(3,2,1)$ et $(2,1,3)$ correspondent à

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}, \qquad \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Chaque ligne et chaque colonne a pour somme un. Si $\mathcal{P}_n$ désigne l'ensemble de ces matrices de permutation, le problème (4.3) équivaut à

$$\min_{P \in \mathcal{P}_n} \langle C, P \rangle, \qquad \langle C, P \rangle = \sum_{i,j} C_{ij} P_{ij}. \tag{4.4}$$

Relâcher la contrainte binaire donne les matrices bistochastiques

$$\mathcal{B}_n = \left\{ P \in \mathbb{R}_+^{n \times n} : \sum_j P_{ij} = 1, \ \sum_i P_{ij} = 1 \right\}. \tag{4.5}$$

La positivité et les sommes des lignes impliquent déjà $P_{ij} \leq 1$. Le problème relaxé est

$$\min_{P \in \mathcal{B}_n} \langle C, P \rangle. \tag{4.6}$$

L'ensemble $\mathcal{B}_n$ est compact et convexe : pour $P, Q \in \mathcal{B}_n$ et $0 \leq t \leq 1$, la matrice $(1-t)P + tQ$ reste admissible. Un minimiseur existe car l'objectif est continu.

Le développement de la programmation linéaire, notamment de la méthode du simplexe de Dantzig [14], a rendu ces problèmes accessibles au calcul. Des algorithmes d'affectation spécialisés, comme certaines implémentations de la méthode hongroise, résolvent le problème dense $n \times n$ en $O(n^3)$ opérations. Cette borne ne doit pas être attribuée à la méthode générale du simplexe, dont le temps de calcul dans le pire cas n'est pas polynomial.

4.2.2 Pourquoi la relaxation est exacte pour des masses égales

Le théorème de Birkhoff–von Neumann affirme que toute matrice bistochastique est une combinaison convexe de matrices de permutation [4, 47] :

$$P = \sum_{k=1}^{K} \lambda_k P^{(k)}, \qquad \lambda_k \geq 0, \quad \sum_k \lambda_k = 1.$$

Ainsi, $\langle C, P \rangle = \sum_k \lambda_k \langle C, P^{(k)} \rangle$ ne peut pas être inférieur au plus petit coût d'une permutation. Comme toute matrice de permutation est admissible pour la relaxation, on obtient

$$\min_{P \in \mathcal{B}_n} \langle C, P \rangle = \min_{P \in \mathcal{P}_n} \langle C, P \rangle. \tag{4.7}$$

Au moins un plan optimal est donc une permutation. Cela ne signifie pas que tout plan optimal soit une permutation : la moyenne de deux permutations optimales distinctes est également optimale. Si tous les coûts sont nuls, toute matrice bistochastique est optimale.

Pour certifier une valeur optimale, on peut chercher des nombres u_i, v_j tels que $u_i + v_j \le C_{ij}$. Toute matrice $P \in \mathcal{B}_n$ vérifie alors

$$\langle C, P\rangle \ge \sum_{i,j}(u_i + v_j)P_{ij} = \sum_i u_i + \sum_j v_j.$$

Dans l'exemple parisien, $u = (2,3,3,0,5,2)$ et $v = (10,4,9,7,8,11)$ vérifient les 36 inégalités et la somme de leurs composantes vaut 64. Avec une affectation de coût 64, ils prouvent donc l'optimalité sans énumérer toutes les permutations.

4.2.3 Des nombres de points et des masses différents

Supposons maintenant que n boulangeries fournissent des quantités $a_i \ge 0$ et que m cafés demandent des quantités $b_j \ge 0$, avec la même masse totale

$$\sum_{i=1}^{n} a_i = \sum_{j=1}^{m} b_j = s.$$

Un plan de transport appartient à

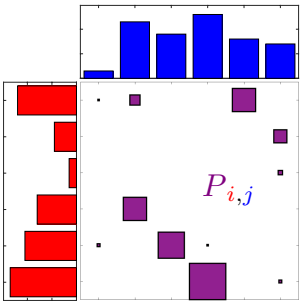
$$\mathcal{B}(a,b) = \left\{ P \in \mathbb{R}_+^{n\times m} : \sum_j P_{ij} = a_i,\ \sum_i P_{ij} = b_j \right\}.$$

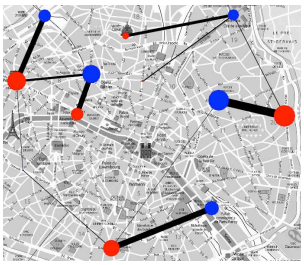
Pour $s > 0$, le plan $P_{ij} = a_i b_j / s$ prouve que cet ensemble est non vide. Pour $s = 0$, le seul plan est nul. Le problème de transport s'écrit

$$\min_{P\in\mathcal{B}(a,b)} \sum_{i=1}^{n}\sum_{j=1}^{m} C_{ij}P_{ij}. \tag{4.8}$$

Pour des coûts finis, la compacité assure encore l'existence d'un minimiseur. L'entrée P_{ij} est la quantité expédiée de i vers j, et C_{ij} son coût par unité de masse.

	3	23	18	26	16	14
24	1	7	0	0	16	0
9	0	0	0	0	0	9
3	0	0	0	0	0	3
16	0	16	0	0	0	0
21	2	0	18	1	0	0
27	0	0	0	25	0	2





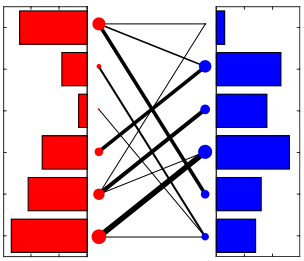


Figure 4.5. Quatre représentations du même couplage : ses entrées matricielles, leurs amplitudes représentées par des carrés, des segments pondérés dans l'espace et un graphe biparti. Les sommes des lignes et des colonnes donnent les masses prescrites a et b.

Le partage de la masse peut devenir nécessaire. Une boulangerie de masse 2 et deux cafés demandant chacun une masse 1 admettent pour seul plan $P = (1\ 1)$. Une application qui envoie toute la masse source vers une seule cible ne peut pas réaliser ces marginales. Un plan de transport est donc plus général qu'une application de transport. Pour des masses égales $a_i = b_j = 1$, on retrouve $\mathcal{B}_n$; pour des masses de probabilité $a_i = b_j = 1/n$, les plans correspondants sont les P/n.

4.3 Mesures et distances de transport

Les mêmes idées s'appliquent aux mesures de probabilité, notamment aux distributions discrètes $\mu = \sum_i a_i \delta_{x_i}$ et $\nu = \sum_j b_j \delta_{y_j}$. Un couplage $\pi \in \Pi(\mu,\nu)$ est une mesure de probabilité sur les couples (x,y) dont

les marginales sont μ et ν. Une application mesurable T induit un couplage $(\mathrm{Id}, T)_{\#}\mu$ lorsque $T_{\#}\mu = \nu$; cette notation signifie qu'appliquer T à une variable aléatoire de loi μ donne une variable de loi ν.

Pour des mesures de probabilité sur $\mathbb{R}^d$ à moments d'ordre deux finis, la distance de transport quadratique est

$$W_2(\mu,\nu)^2 = \min_{\pi\in\Pi(\mu,\nu)} \int |x-y|^2 \,\mathrm{d}\pi(x,y). \tag{4.9}$$

Dans le cas discret, on retrouve (4.8) avec $C_{ij} = |x_i - y_j|^2$. La distance dépend des masses et des positions : des vecteurs de poids égaux placés à des endroits différents ne décrivent pas nécessairement la même mesure. La racine carrée W_2 est une distance, et $W_2(\mu,\nu) = 0$ exactement lorsque $\mu = \nu$. Son carré ne vérifie généralement pas l'inégalité triangulaire : pour des masses de Dirac en 0, 1 et 2, les distances au carré correspondantes valent 1, 1 et 4.

Le théorème de Brenier précise une condition sous laquelle le couplage optimal est une application [5]. Si la source μ est absolument continue par rapport à la mesure de Lebesgue et si les deux mesures ont des moments d'ordre deux finis, le problème quadratique admet un unique couplage optimal. Il est induit par une application $T = \nabla\varphi$, où φ est convexe, et T est unique μ-presque partout. L'absolue continuité et le choix du coût sont essentiels ; on ne peut pas affirmer cette conclusion pour des mesures et des coûts arbitraires.

4.4 Applications

4.4.1 Transférer une palette de couleurs

Pour deux images ayant le même nombre n de pixels, notons $x_i, y_j \in \mathbb{R}^3$ leurs couleurs RVB. Une affectation minimisant $\sum_i |x_i - y_{\sigma(i)}|^2$ donne une image dont la couleur à la position source i est $y_{\sigma(i)}$. Chaque couleur cible est utilisée une fois : la sortie possède donc exactement la distribution empirique des couleurs de la cible. L'optimisation limite les changements de couleur, mais ne comporte aucun terme de régularité spatiale ; la préservation des contours et des textures n'est donc pas garantie.



Figure 4.6. Transfert de palette. Colonnes : source, cible et source recolorée. En haut, les couleurs sur la grille de l'image ; en bas, les nuages de points correspondants dans l'espace RVB. Une permutation des couleurs cibles donne le même nuage dans les deux dernières colonnes.

Cette distinction sépare le transport de couleurs du déplacement des pixels dans le plan de l'image. Ici, les positions transportées sont des points de l'espace des couleurs ; la grille d'affichage reste fixe. Pour des images de tailles différentes ou des groupes de couleurs pondérés, on utilise un couplage et une règle précisant sa conversion en couleurs de sortie. Une moyenne barycentrique des couleurs couplées ne préserve pas nécessairement l'histogramme cible exact.

4.4.2 Barycentres de formes

La moyenne euclidienne pondérée de trois points minimise $\alpha|x-r|^2+\beta|y-r|^2+\gamma|z-r|^2$, où $\alpha,\beta,\gamma\geq 0$ et $\alpha+\beta+\gamma=1$. Elle vaut $r=\alpha x+\beta y+\gamma z$. Remplacer les distances euclidiennes au carré par des distances de transport au carré définit un barycentre de Wasserstein [2] :

$$\min_{\rho\in\mathcal{P}_2(\mathbb{R}^d)} \alpha W_2(\mu,\rho)^2+\beta W_2(\nu,\rho)^2+\gamma W_2(\eta,\rho)^2. \tag{4.10}$$

On note $\mathcal{P}_2$ l'ensemble des mesures de probabilité à moment d'ordre deux fini. Dans un calcul sur une grille, on restreint plutôt ρ à des poids de probabilité sur cette grille. Une forme se représente par une densité uniforme à l'intérieur, normalisée pour avoir une masse totale égale à un. Faire varier les trois poids produit une famille de distributions interpolées (figure 4.7). Ces distributions ne sont pas nécessairement uniformes à l'intérieur d'une forme au contour net, et leur unicité demande des hypothèses supplémentaires.

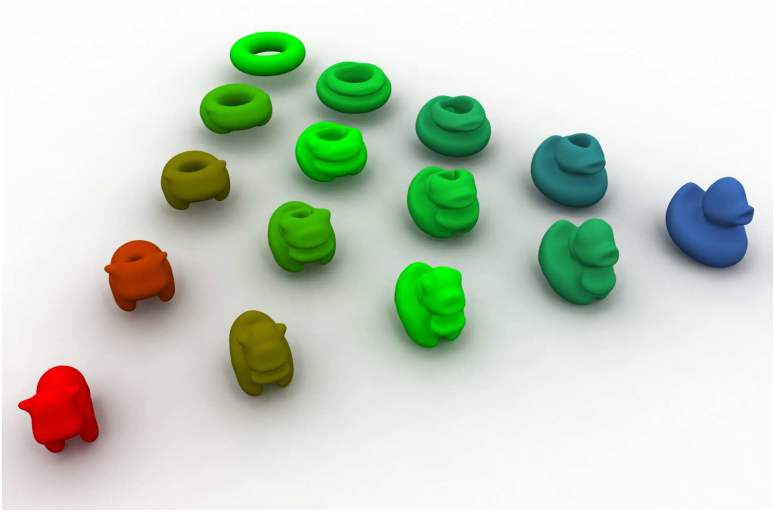


Figure 4.7. Interpolation barycentrique numérique de formes tridimensionnelles représentées par des distributions de probabilité. Les trois formes de référence occupent les sommets du triangle ; les points intérieurs correspondent à différents poids dans (4.10).

4.4.3 Comparer des textes

Normaliser les nombres d'occurrences des mots d'un texte donne une distribution de probabilité. Le transport compare deux telles distributions à l'aide d'un coût décrivant la dissimilarité entre mots, par exemple une distance entre leurs plongements vectoriels. Le choix de ce coût détermine quelles substitutions sont peu coûteuses ; on peut aussi l'apprendre pour une tâche donnée [21]. Les histogrammes de mots oublient leur ordre et ne peuvent donc pas, à eux seuls, saisir toutes les distinctions grammaticales ou sémantiques.



Figure 4.8. Nuages des mots les plus fréquents de deux textes. Leur taille indique leur fréquence relative ; ces illustrations ne constituent pas un histogramme numérique complet.

Pour aller plus loin. Pour la théorie, on pourra consulter [39, 46]. Les méthodes numériques et les applications sont développées dans [35]. Le cours Optimal Transport for Machine Learning (OT4ML) propose une autre introduction à ces idées.

CHAPITRE 5

Réseaux de neurones

Les réseaux de neurones apprennent des transformations à partir d'exemples. Ce chapitre explique comment composer et entraîner leurs couches, puis comment un générateur peut apprendre à reproduire une distribution de données. Le lien avec le transport optimal éclaire les objectifs de cet apprentissage et ses limites.

5.1 Réseaux discriminatifs

5.1.1 Une fonction construite par couches

Un réseau de neurones est une fonction paramétrée f_w. À partir d'une entrée x, par exemple une image, il calcule une sortie $\hat{y} = f_w(x)$. Pour la classification, la sortie peut représenter des probabilités estimées de plusieurs classes. Une probabilité élevée exprime la confiance du modèle ; elle ne garantit ni que cette confiance soit bien calibrée, ni que la prédiction soit correcte.

Supposons d'abord qu'une image appartienne à exactement l'une de deux classes : chat ou chien. Un classifieur peut renvoyer deux nombres positifs ou nuls de somme un. Cela diffère d'une tâche à étiquettes multiples, où une image peut contenir à la fois un chat et un chien : dans ce cas, les probabilités séparées n'ont pas à sommer à un.

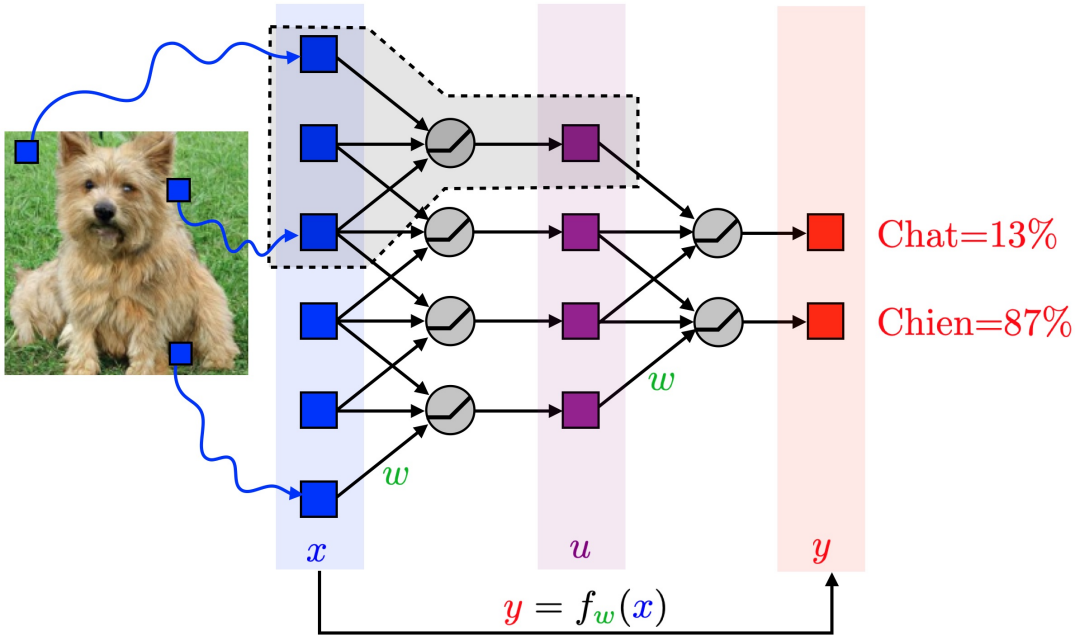


FIGURE 5.1. Schéma d'un classifieur avec une couche cachée et une couche de sortie. Seuls quelques pixels d'entrée et quelques connexions sont dessinés. Les pourcentages sont des prédictions du modèle ; une normalisation telle que softmax est nécessaire pour obtenir des probabilités de classes.

Chaque couche associe des opérations affines à une fonction d'activation. Une unité linéaire rectifiée, ou ReLU, calcule

$$u_1 = \mathrm{ReLU}(w_1 x_1 + w_2 x_2 + w_3 x_3 + b), \qquad \mathrm{ReLU}(t) = \max(t, 0). \tag{5.1}$$

Les poids déterminent la contribution de chaque entrée ; le biais b déplace le seuil d'activation. Une ReLU renvoie zéro pour une entrée négative ou nulle et conserve les entrées positives. Elle diffère d'un neurone à

seuil binaire, qui n'a que deux valeurs de sortie possibles.

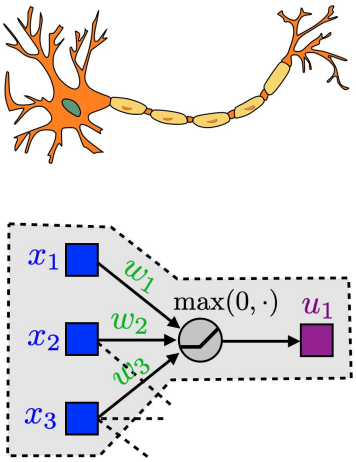


Figure 5.2. Un neurone biologique et un neurone artificiel schématique. Le modèle mathématique ne conserve qu'une somme pondérée et une activation ; le biais de (5.1) est omis du dessin.

Pour une entrée $x \in \mathbb{R}^d$, une couche cachée de largeur m et K classes, on peut définir le réseau

$$u = \mathrm{ReLU}(Wx + b), \qquad a = Vu + c, \qquad \hat{y}_k = \frac{e^{a_k}}{\sum_{j=1}^{K} e^{a_j}},$$

où $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$, $V \in \mathbb{R}^{K \times m}$ et $c \in \mathbb{R}^K$. ReLU agit coordonnée par coordonnée. L'opération finale est softmax ; les valeurs intermédiaires a_k sont appelées *logits*. En calcul numérique, remplacer chaque a_k par $a_k - \max_j a_j$ donne les mêmes probabilités et évite les débordements. Les entrées de W, b, V, c forment le vecteur de paramètres w.

Composer davantage de couches donne un réseau profond. Les activations non linéaires sont essentielles : une composition d'applications affines reste affine. La profondeur permet de représenter certaines fonctions plus économiquement qu'une seule couche cachée très large, mais l'architecture appropriée dépend de la tâche.

Le vocabulaire biologique exprime une analogie, et non un modèle fidèle du cerveau. McCulloch et Pitts ont étudié des neurones logiques à seuil [31], tandis que le perceptron de Rosenblatt a introduit un premier modèle d'apprentissage [37]. Ces modèles se distinguent des réseaux ReLU décrits ici. Les études biologiques du cortex visuel et de la dynamique membranaire portent sur des mécanismes bien plus riches [20, 22].

5.1.2 Apprendre à partir d'exemples étiquetés

L'apprentissage supervisé part de couples $(x_i, y_i)_{i=1}^n$. Pour une étiquette de classe unique k_i, la cible est souvent un vecteur indicateur : $(y_i)_{k_i} = 1$, et toutes les autres composantes sont nulles. Ces étiquettes représentent des annotations, plutôt que des probabilités conditionnelles connues. Les données peuvent provenir d'annotations humaines ou d'une autre source de supervision, et contenir des erreurs.

L'objectif d'apprentissage mesure le désaccord entre prédictions et étiquettes. L'erreur quadratique moyenne et l'entropie croisée en sont deux exemples :

$$E_{\mathrm{sq}}(w) = \frac{1}{n} \sum_{i=1}^{n} \| f_w(x_i) - y_i \|_2^2, \tag{5.2}$$

$$E_{\mathrm{CE}}(w) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} (y_i)_k \log(f_w(x_i)_k). \tag{5.3}$$

Pour une cible indicatrice, le terme d'entropie croisée correspondant vaut $-\log(f_w(x_i)_{k_i})$. Le logarithme est naturel, et softmax donne des probabilités strictement positives pour des logits finis. Une implémentation stable évalue directement le logarithme de softmax. On peut ajouter à l'objectif une pénalité sur les poids.

L'entraînement cherche à réduire cette erreur empirique. Comme le réseau dépend non linéairement de nombreux paramètres, l'objectif est généralement non convexe. Les méthodes d'optimisation peuvent être efficaces sans fournir de garantie générale d'atteindre son minimum global.



Figure 5.3. Exemples d'images étiquetées de chiens et de chats issues d'ImageNet [15]. L'apprentissage utilise de nombreux exemples ; l'évaluation demande des données séparées.

5.1.3 Rétropropagation et mise à jour des poids

La rétropropagation calcule le gradient d'une perte scalaire en appliquant la règle de dérivation des fonctions composées à travers le réseau, dans l'ordre inverse [28, 38]. C'est la différentiation automatique en mode inverse. Un algorithme d'optimisation utilise ensuite ces dérivées pour mettre à jour les poids ; par exemple, la descente de gradient effectue le pas

$$w^{(t+1)} = w^{(t)} - \eta_t \nabla E(w^{(t)}), \qquad \eta_t > 0.$$

Le choix du pas est important : un pas trop grand peut augmenter l'erreur. La descente de gradient stochastique remplace le gradient complet par une estimation calculée sur un petit lot choisi aléatoirement, ce qui réduit le coût de chaque mise à jour. Un pas stochastique individuel ne diminue pas nécessairement l'erreur totale d'apprentissage.

Pour le classifieur à deux couches ci-dessus et un exemple de perte ℓ donnée par l'entropie croisée, la dérivation se lit directement. Posons $r = Wx + b$, $u = \mathrm{ReLU}(r)$ et $\delta = \hat{y} - y$. En dehors des points où $r_j = 0$,

$$\frac{\partial \ell}{\partial V} = \delta u^{\mathsf{T}}, \qquad \frac{\partial \ell}{\partial c} = \delta, \qquad q = (V^{\mathsf{T}}\delta) \odot \mathbf{1}_{r>0}, \qquad \frac{\partial \ell}{\partial W} = q x^{\mathsf{T}}, \quad \frac{\partial \ell}{\partial b} = q.$$

Le symbole $\odot$ désigne le produit coordonnée par coordonnée. Au point anguleux de ReLU, les logiciels choisissent une convention, souvent zéro, pour la dérivée de l'activation. Cette convention ne rend pas le réseau différentiable au sens classique en ce point.

De grandes bases étiquetées, le calcul efficace sur processeurs graphiques et les progrès des architectures et de l'entraînement ont contribué au succès des classifieurs profonds, notamment au résultat ImageNet de 2012 [25]. Des techniques de régularisation comme le *dropout* peuvent limiter le surapprentissage [43]. Les travaux de Yoshua Bengio, Geoffrey Hinton et Yann LeCun ont été distingués par le prix Turing 2018 de l'ACM, annoncé en 2019. [1]

5.1.4 Approximation, optimisation et généralisation

Un théorème d'approximation universelle porte sur ce qu'un réseau peut représenter. Un résultat classique de Cybenko [13] affirme que, pour une fonction continue $F : [0,1]^d \to \mathbb{R}$ et tout $\varepsilon > 0$, il existe un entier fini m et des coefficients tels que

$$G(x) = \sum_{j=1}^{m} \alpha_j \sigma(v_j \cdot x + \beta_j), \qquad \sup_{x \in [0,1]^d} |F(x) - G(x)| < \varepsilon,$$

où σ est une activation sigmoïde continue fixée, par exemple $\sigma(t) = 1/(1+e^{-t})$. La couche de sortie est linéaire, et la largeur et les poids ne sont pas contraints. Il s'agit du théorème pour les sigmoïdes ; les résultats d'approximation pour les réseaux ReLU demandent leurs propres énoncés.

Ce théorème ne fait intervenir aucune donnée d'apprentissage. Il n'exige pas une infinité d'exemples et ne garantit pas qu'un algorithme puisse trouver le réseau approchant à partir d'un nombre fini d'observations.

1. https://awards.acm.org/about/2018-turing

Trois questions doivent être distinguées : l'existence d'une bonne représentation, la capacité de l'optimisation à la trouver et la qualité des prédictions sur de nouvelles données.

La généralisation s'évalue sur des données exclues de l'entraînement. Les données de validation guident des choix comme l'architecture et le moment d'arrêt ; un jeu de test distinct évalue la procédure finale. La qualité des résultats dépend aussi de la représentativité de ces données par rapport aux conditions d'utilisation du réseau.

Les réseaux convolutifs intègrent des hypothèses sur la structure spatiale [26]. Les connexions locales limitent le champ réceptif, et le partage des poids applique le même filtre en différentes positions. Une convolution de pas un commute avec les translations sur une grille infinie ou périodique ; les conditions au bord et le sous-échantillonnage peuvent modifier cette propriété. L'équivariance par translation d'une carte de caractéristiques se distingue également de l'invariance par translation d'une classification finale.

L'architecture, la régularisation, les données et l'optimisation influencent toutes les performances [27]. Des résultats mathématiques expliquent certains mécanismes et certains régimes, mais aucun théorème unique ne rend compte de toutes les architectures efficaces.

5.2 Réseaux génératifs

5.2.1 Des entrées aléatoires à une distribution

Un générateur g_w transforme une entrée aléatoire $Y \in \mathbb{R}^r$ en une image $X = g_w(Y) \in \mathbb{R}^d$. La loi d'entrée η, par exemple une gaussienne standard, est choisie à l'avance. La loi de sortie est la mesure image $\mu_w = (g_w)_{\#}\eta$. Cette notation décrit la redistribution de la masse de probabilité par le réseau. Un générateur est une transformation apprise, qui n'est généralement pas l'inverse d'un classifieur.

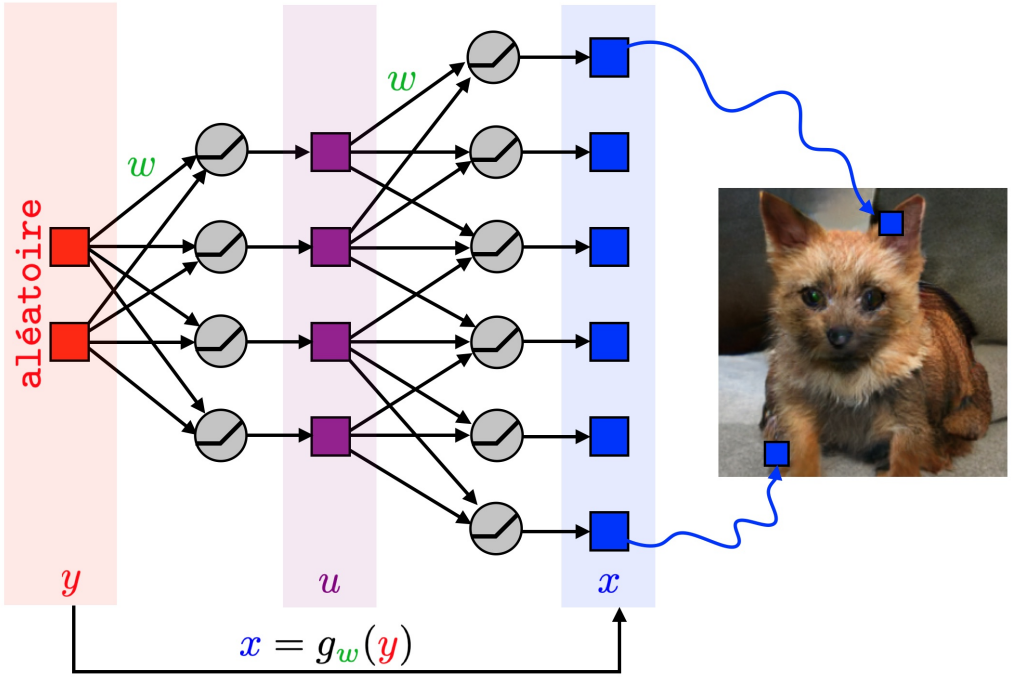


Figure 5.4. Un générateur schématique transforme une entrée aléatoire y, en passant par une couche cachée, en une image $x = g_w(y)$. Les générateurs d'images utilisés en pratique ont davantage de couches et une dimension de sortie bien plus grande.

Pour la génération non conditionnelle, l'apprentissage peut utiliser des images sans étiquette de classe. Leur collecte, leur sélection et leur prétraitement demandent toujours de l'attention. La génération conditionnelle fournit en outre une information telle qu'une étiquette de classe, donnant un modèle $g_w(y, c)$. Tous les modèles génératifs ne sont donc pas entraînés sans supervision.

Le but est de reproduire une distribution, y compris sa diversité. Si la distribution de référence attribue une probabilité $1/2$ aux chats et $1/2$ aux chiens, un générateur idéal possède aussi ces probabilités. Un lot aléatoire fini ne contient pas nécessairement exactement la moitié de chaque catégorie. Un modèle produisant toujours

la même image plausible n'a pas appris toute la distribution ; c'est une forme extrême d'effondrement des modes.

5.2.2 Comparer des distributions empiriques

Notons $z_1, \dots, z_n$ les images d'apprentissage et $x_i = g_w(y_i)$ les images générées à partir d'échantillons latents indépendants. Leurs distributions empiriques sont

$$\mu_{w,n} = \frac{1}{n} \sum_{i=1}^{n} \delta_{x_i}, \qquad \nu_n = \frac{1}{n} \sum_{j=1}^{n} \delta_{z_j}.$$

L'emploi de mesures conserve les fréquences et les répétitions d'échantillons, que la notation ensembliste ordinaire effacerait. Un objectif possible est une distance de transport entre ces mesures.

Pour un coût euclidien, la distance de Wasserstein empirique d'ordre un vaut

$$W_1(\mu_{w,n}, \nu_n) = \frac{1}{n} \min_{s \in \Sigma_n} \sum_{i=1}^{n} \|g_w(y_i) - z_{s(i)}\|_2. \tag{5.4}$$

Le facteur $1/n$ normalise la masse totale de probabilité. Cette distance diffère du coût quadratique introduit au chapitre sur le transport. Pour des nombres d'échantillons ou des poids différents, on remplace la permutation par un couplage aux marginales prescrites.

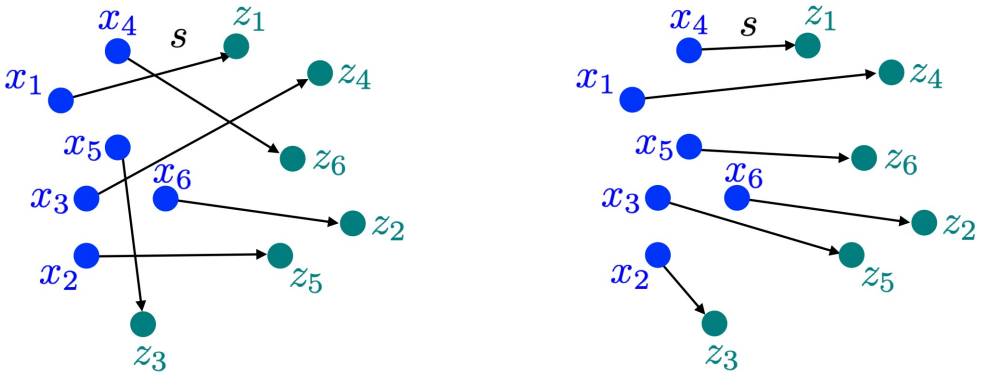


FIGURE 5.5. Deux affectations de six points sources à six points cibles. L'affectation de droite a un coût de transport euclidien plus faible et constitue l'appariement optimal de cet exemple.

À la figure 5.5, l'affectation de gauche est $(1, 5, 4, 6, 3, 2)$, tandis que celle de droite est $(4, 3, 5, 1, 6, 2)$. La notation donne la liste $(s(1), \dots, s(6))$. Bien qu'il y ait $6! = 720$ possibilités, des algorithmes d'affectation efficaces évitent l'énumération des $n!$ permutations et résolvent un problème dense en $O(n^3)$ opérations. Le chapitre sur le transport explique la relaxation convexe sous-jacente [35].

L'objectif d'apprentissage distributionnel est $\min_w W_1(\mu_w, \nu)$ pour une loi cible ν. Le transport entre échantillons finis en fournit une approximation numérique, mais soulève des difficultés statistiques et d'optimisation. Le transport entre petits lots est généralement un estimateur biaisé de la distance entre les lois. L'affectation optimale peut changer avec w, de sorte que l'objectif empirique n'est pas partout différentiable.

La distance de base joue aussi un rôle essentiel. La distance euclidienne entre tableaux de pixels est sensible aux petites translations et peut mal refléter la similarité sémantique. Une représentation par caractéristiques peut fournir un coût plus pertinent, mais son choix modifie la géométrie apprise.

5.2.3 Réseaux génératifs antagonistes

Les réseaux génératifs antagonistes, ou GAN, introduits par Goodfellow et ses collaborateurs en 2014 [19], entraînent un générateur face à un discriminateur. Le discriminateur $D_v(x) \in (0, 1)$ cherche à distinguer les données des échantillons générés. L'objectif minimax original est

$$\min_w \max_v \left\{ \mathbb{E}_{Z \sim \nu}[\log D_v(Z)] + \mathbb{E}_{Y \sim \eta}[\log(1 - D_v(g_w(Y)))] \right\}. \tag{5.5}$$

Le discriminateur augmente l'objectif ; le générateur le diminue. Des mises à jour de gradient alternées approchent ce jeu, sans garantie générale de convergence pour des réseaux arbitraires.

Pour un discriminateur sans restriction et optimisé exactement, le supremum intérieur vaut $-\log 4 + 2\,\mathrm{JS}(\nu, \mu_w)$, où la divergence de Jensen–Shannon est

$$\mathrm{JS}(\nu, \mu) = \tfrac{1}{2}\,\mathrm{KL}(\nu \mid m) + \tfrac{1}{2}\,\mathrm{KL}(\mu \mid m), \qquad m = \tfrac{1}{2}(\nu + \mu).$$

Ici, KL utilise les logarithmes naturels et se définit entre mesures à l'aide de leurs densités de Radon–Nikodym. Les deux mesures sont absolument continues par rapport à m. L'identité concerne l'optimum idéal du discriminateur ; des réseaux finis et une optimisation incomplète modifient l'objectif effectivement atteint. En pratique, on utilise souvent pour le générateur la perte $-\mathbb{E}\log D_v(g_w(Y))$, qui vise la même distribution optimale mais donne des gradients différents.

5.2.4 Le lien avec Wasserstein

Les GAN de Wasserstein [3] emploient un autre objectif, fondé sur la formule duale de Kantorovitch–Rubinstein. Pour des mesures de probabilité sur $\mathbb{R}^d$ à moments d'ordre un finis,

$$W_1(\nu, \mu_w) = \sup_{\mathrm{Lip}(h)\le 1} \left\{ \mathbb{E}_{Z\sim\nu} h(Z) - \mathbb{E}_{Y\sim\eta} h(g_w(Y)) \right\}. \tag{5.6}$$

La contrainte signifie $|h(x) - h(x')| \le \|x - x'\|_2$ pour tous x, x'. Un réseau critique approche cette classe de fonctions ; ses valeurs sont des scores réels, et non des probabilités de classes. Le générateur minimise l'écart estimé par le critique.

Le critique est un potentiel dual pour une distance de base *fixée* ; il n'apprend pas automatiquement un nouveau coût de transport entre paires d'images. Imposer ou approcher la contrainte lipschitzienne est essentiel. La restriction de la classe des critiques et leur optimisation approchée signifient aussi que le score calculé n'est pas nécessairement égal à la véritable distance de Wasserstein.

Les GAN ordinaires et les GAN de Wasserstein expriment ainsi des principes antagonistes proches, avec des objectifs mathématiques différents. Aucune des deux procédures ne garantit que toute image générée soit pertinente ou que tous les modes soient représentés.



Figure 5.6. Deux suites d'images synthétiques interpolant entre chats et chiens, obtenues avec BigGAN [6]. BigGAN est conditionné par les classes : cette information fait partie du conditionnement. Ces exemples illustrent la génération et l'interpolation, sans affirmer qu'il s'agit de chemins de transport optimal.

Interpoler les variables latentes ou de conditionnement peut produire des transitions visuelles régulières, comme à la figure 5.6. Ces chemins ne sont pas nécessairement des géodésiques de transport et ne restent pas toujours typiques de la distribution d'apprentissage. La plausibilité visuelle, la diversité, la fidélité à la loi cible et les résultats sur des données réservées sont des aspects distincts de la qualité d'un modèle génératif.

Glossaire

Les termes introduits dans les deux premiers chapitres sont réunis ici pour faciliter leur consultation.

Information et codage

- **Pixel** : emplacement dans la grille d'une image, ou valeur qui y est stockée.
- **Symbole** : élément de l'alphabet fini à coder.
- **Mot de code** : suite de bits associée à un symbole.
- **Code** : correspondance entre les symboles et leurs mots de code.
- **Distribution empirique** : fréquences relatives observées des symboles.
- **Histogramme** : représentation graphique de ces fréquences.
- **Modèle de source** : loi de probabilité des symboles ou de leurs suites.
- **Entropie** : information moyenne $H(V)$, positive ou nulle, mesurée en bits.
- **Longueur moyenne du code** : $L(c) = \sum_v p_v \ell_v$, en bits par symbole.

Traitement des images

- **Bit et octet** : un chiffre binaire ; un groupe de huit bits.
- **Canal** : l'un des tableaux de composantes d'une image multicanal.
- **Quantification** : remplacement des valeurs par des indices d'un ensemble fini.
- **Bruit** : fluctuations indésirables dues à l'acquisition ou au traitement.
- **Moyenne** : somme des valeurs divisée par leur nombre.
- **Médiane** : valeur centrale après classement d'un nombre impair de valeurs.
- **Contour** : variation spatiale rapide des valeurs de l'image.
- **Flou** : perte de transitions nettes et de détails fins.
- **Contraste** : différences entre les régions claires et les régions sombres.
- **Luminance** : grandeur photométrique définie à partir de la sensibilité visuelle.
- **Matrice** : tableau rectangulaire $(a_{i,j})$.
- **Résolution** : terme qui peut désigner les dimensions en pixels ou le pouvoir de séparation spatial ; ce sont deux notions distinctes.
- **Fondu** : moyenne pondérée de deux images à positions de pixels fixes.

Références

[1] B. Adcock, A. C. Hansen, C. Poon, and B. Roman. Breaking the coherence barrier : asymptotic incoherence and asymptotic sparsity in compressed sensing. *CoRR*, abs/1302.0561, 2013. URL http://arxiv.org/abs/1302.0561.

[2] Martial Agueh and Guillaume Carlier. Barycenters in the wasserstein space. *SIAM Journal on Mathematical Analysis*, 43(2) :904–924, 2011.

[3] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223, 2017. URL https://proceedings.mlr.press/v70/arjovsky17a.html.

[4] Garrett Birkhoff. Tres observaciones sobre el álgebra lineal. *Universidad Nacional de Tucumán Revista Series A*, 5 :147–151, 1946.

[5] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on Pure and Applied Mathematics*, 44(4) :375–417, 1991.

[6] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1809.11096.

[7] E. J. Candès, J. Romberg, and T. Tao. Stable signal recovery from incomplete and inaccurate measurements. *Communications on Pure and Applied Mathematics*, 59(8) :1207–1223, 2006.

[8] E.J. Candès and C. Fernandez-Granda. Towards a mathematical theory of super-resolution. *Communications on Pure and Applied Mathematics*, 67(6) :906–956, 2014. ISSN 1097-0312. doi : 10.1002/cpa.21455. URL http://dx.doi.org/10.1002/cpa.21455.

[9] N. Chauffert, P. Ciuciu, J. Kahn, and P. Weiss. Variable density sampling with continuous trajectories. *SIAM Journal on Imaging Sciences*, 7(4) :1962–1992, 2014.

[10] S. S. Chen, D. L. Donoho, and M. A. Saunders. Atomic decomposition by basis pursuit. *SIAM Journal on Scientific Computing*, 20(1) :33–61, 1998. doi : 10.1137/S1064827596304010.

[11] Patrick L. Combettes and Jean-Christophe Pesquet. Proximal splitting methods in signal processing. In *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*, pages 185–212. Springer, 2011. doi : 10.1007/978-1-4419-9569-8_10.

[12] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2 edition, 2006.

[13] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4) :303–314, 1989.

[14] George B. Dantzig. Application of the simplex method to a transportation problem. In Tjalling C. Koopmans, editor, *Activity Analysis of Production and Allocation*, pages 359–373. Wiley, 1951.

[15] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet : A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.

[16] D. L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4) :1289–1306, 2006.

[17] M. F. Duarte, M. A. Davenport, D. Takhar, J. N. Laska, T. Sun, K. F. Kelly, and R. G. Baraniuk. Single-pixel imaging via compressive sampling. *IEEE Signal Processing Magazine*, 25(2) :83–91, March 2008. ISSN 1053-5888.

[18] V. Duval and G. Peyré. Exact support recovery for sparse spikes deconvolution. *Foundations of Computational Mathematics*, 15(5) :1315–1355, 2015. doi : 10.1007/s10208-014-9228-6. URL http://hal.archives-ouvertes.fr/hal-00839635/.

[19] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014.

[20] Alan L Hodgkin and Andrew F Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of physiology*, 117(4) :500–544, 1952.

[21] Gao Huang, Chuan Guo, Matt J Kusner, Yu Sun, Fei Sha, and Kilian Q Weinberger. Supervised word mover's distance. In *Advances in Neural Information Processing Systems*, pages 4862–4870, 2016.

[22] David H Hubel and Torsten N Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *The Journal of physiology*, 160(1) :106–154, 1962.

[23] D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the Institute of Radio Engineers*, 40(9) :1098–1101, 1952.

[24] Leonid Kantorovich. On the transfer of masses (in russian). *Doklady Akademii Nauk*, 37(2) :227–229, 1942.

[25] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.

[26] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11) :2278–2324, 1998.

[27] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553) :436–444, 2015.

[28] Seppo Linnainmaa. Taylor expansion of the accumulated rounding error. *BIT Numerical Mathematics*, 16 (2) :146–160, 1976.

[29] S. G. Mallat. *A Wavelet Tour of Signal Processing : The Sparse Way*. Elsevier/Academic Press, Amsterdam, third edition, 2009.

[30] S. G. Mallat and Z. Zhang. Matching pursuits with time-frequency dictionaries. *IEEE Transactions on Signal Processing*, 41(12) :3397–3415, 1993.

[31] Warren S McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4) :115–133, 1943.

[32] Gaspard Monge. Mémoire sur la théorie des déblais et des remblais. *Histoire de l'Académie Royale des Sciences*, pages 666–704, 1781.

[33] B. K. Natarajan. Sparse approximate solutions to linear systems. *SIAM Journal on Computing*, 24(2) : 227–234, 1995. doi : 10.1137/S0097539792240406.

[34] Gabriel Peyré. The numerical tours of signal processing. *Computing in Science & Engineering*, 13(4) :94–97, 2011. doi : 10.1109/MCSE.2011.71. URL https://www.numerical-tours.com/.

[35] Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6) :355–607, 2019. URL https://arxiv.org/abs/1803.00567.

[36] J. Rissanen and G. Langdon. Arithmetic coding. *IBM Journal of Research and Development*, 23(2) :149–162, 1979.

[37] Frank Rosenblatt. The perceptron : A perceiving and recognizing automaton. Technical Report 85-460-1, Cornell Aeronautical Laboratory, 1957.

[38] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088) :533–536, 1986.

[39] Filippo Santambrogio. *Optimal transport for applied mathematicians*. Birkhäuser, 2015.

[40] F. Santosa and W.W. Symes. Linear inversion of band-limited reflection seismograms. *SIAM Journal on Scientific and Statistical Computing*, 7(4) :1307–1330, 1986. doi : 10.1137/0907087.

[41] C. E. Shannon. A Mathematical Theory of Communication. *The Bell System Technical Journal*, 27(3) : 379–423, 1948.

[42] C. E. Shannon. Communication in the presence of noise. *Proc. Institute of Radio Engineers*, 37(1) :10–21, 1949.

[43] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout : a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1) : 1929–1958, 2014.

[44] R. Tibshirani. Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society. Series B. Methodological*, 58(1) :267–288, 1996.

[45] Samuel Vaiter, Gabriel Peyré, and Jalal Fadili. Low complexity regularization of linear inverse problems. In Götz E. Pfander, editor, *Sampling Theory, a Renaissance*, pages 103–153. Birkhäuser, 2015. URL https://hal.science/hal-01018927.

[46] Cédric Villani. *Topics in Optimal Transportation*. Graduate Studies in Mathematics Series. American Mathematical Society, 2003. ISBN 9780821833124.

[47] John Von Neumann. A certain zero-sum two-person game equivalent to the optimal assignment problem. *Contributions to the Theory of Games*, 2 :5–12, 1953.